Department of Computer Science

Bachelor's thesis

Deep reinforcement learning for adaptive shot allocation in quantum circuit execution

Supervisors:
Prof. Antonio Brogi
Dr. Alessandro Bocci
Dr. Giuseppe Bisicchia

Candidate:
Sergio De Crescenzo

Academic Year 2024/2025

Contents

Acknowledgements **1**

1 Introduction **2**

1.1 Context 2

1.2 Problem Statement 5

1.3 Objectives 5

1.4 Thesis structure 6

2 Background **8**

2.1 Quantum Computing 8

2.1.1 The qubit: beyond 0 and 1 9

2.1.2 Definition of a Quantum State 11

2.1.3 Measurement: the bridge to the classical world 12

2.1.4 Entanglement: a deeper connection 14

2.1.5 Noise, decoherence, and the NISQ era 15

2.2 The shot allocation problem in Quantum Computing 17

2.2.1 Incremental Execution: a convergence-based approach 18

2.3 Reinforcement Learning agents 23

2.3.1 The principles of Reinforcement Learning 24

2.3.2 Maximizing cumulative reward 26

2.3.3 Policies and value functions 26

2.3.4 Exploration vs. exploitation 27

2.3.5 Q-Learning 27

2.3.6 Deep Q-Learning (DQN) 29

2.4 Understanding Neural Networks 32

2.4.1 Architecture: layers and neurons 32

2.4.2 Weights and biases: the learnable parameters 33

2.4.3 Activation Functions: Introducing Non-linearity 33
2.4.4 Forward Propagation: Making Predictions 34
2.4.5 Learning: backpropagation and gradient descent 34
2.4.6 The black box revisited 35

3 Used Technologies **36**

4 Agent **39**
4.1 System architecture overview 41
4.2 Diving in agent's perception: the state space 42
4.3 The agent's brain: Deep Q-Network architecture 48
4.3.1 Network Structure 49
4.4 Defining the learning process 51
4.4.1 Architecture and initialization 51
4.4.2 Action selection and exploration 52
4.4.3 Experience replay mechanism and Bellman update with target networks 53
4.4.4 Network synchronization 55
4.5 The training regimen 56
4.5.1 Training Data and Environment Setup 56
4.5.2 The reward structure 57
4.5.3 Procedural steps: the DQN learning cycle 60

5 Validation **63**
5.1 Experimental Framework 63
5.1.1 Dataset and problem space 63
5.1.2 Data handling and sampling strategy 66
5.1.3 Evaluation metrics 67
5.2 Experimental results 71
5.2.1 2,000 episodes configuration 76
5.2.2 3,000 episodes configuration 79

5.2.3 5,000 episodes configuration 82
5.3 Experiments discussion and critical analysis 85
5.3.1 The optimal balance: 3000 episodes 86
5.3.2 Critical analysis of policy degradation 87
5.3.3 Strengths and behavioral characteristics 87

6 Conclusions **89**
6.1 Summary of Thesis Work 89
6.2 Reached objectives and validation of results 91
6.3 Possible future works 92
6.4 Reflections and skills acquired 93

List of Figures

2.1 Up and down spins of an electron 9

2.2 The Bloch sphere representation of a qubit state 11

2.3 Double slit experiment, observed system vs not observed system. 13

2.4 Stern-Gerlach experiment elements. The photographic plate on the right is used to observe the final beam pattern. 14

2.5 Error increasing the number of shots. [2] 19

2.6 The Total Variation Distance (TVD) between consecutive cumulative distributions as a function of the number of shots. Each line represents a different circuit-backend pair. The distance decreases sharply at first and then plateaus, signaling that further shots provide diminishing returns. [2] 22

2.7 Machine Learning three broad categories [15] 24

2.8 The Agent-Environment Interaction Loop 25

2.9 Gridworld environment example (left) and corresponding Q-table mapping states to actions (right). In this example the agent's policy is to select the action with the highest Q-value starting from the top-left cell(indicated by the arrow in the left table). 28

2.10 To handle large state spaces, the discrete Q-table is replaced by a function that maps a state-action pair to a Q-value. 29

2.11 Deep Q-Network Architecture 30

2.12 Target Network 31

4.1 High-level architecture of the reinforcement learning system for adaptive shot allocation. The agent observes states and takes actions iteratively until it decides to stop. Only at episode termination the environment consults the Oracle to evaluate the agent's performance and compute the final reward. The Oracle's optimal shot count is never revealed to the agent during its decision-making process. 42

4.2 Detailed neuron-level view of the DQN architecture. Only representative neurons are shown for each layer; actual layer sizes are indicated in parentheses. 50

5.1 Snapshots of the JSON dataset structure: Shot configuration, Experiment metadata, and Measurement data. 65
5.2 Example of reward evolution. The agent improves over time (orange trend) despite the noise inherent in the environment (light blue). 68
5.3 Example of MAE progression. A downward trend indicates the agent is learning to stop closer to the optimal point. 69
5.4 Example scatter plot. Points clustering around the diagonal indicate high accuracy; points significantly above indicate systematic overshooting. . . 70
5.5 Example histogram. Ideally, the mass is centered at 0. A long tail to the right is preferable to a tail to the left in this domain. 70
5.6 Example of stratified errors. This chart helps identify if the agent fails specifically on complex tasks. 71
5.7 Evolution of episodic rewards over 10,000 training episodes. The semi-transparent blue area represents raw episodic rewards, while the orange line shows the 200-episode moving average, revealing underlying learning trends. 73
5.8 Mean Absolute Error during 10,000 training episodes. The semi-transparent green area shows episode-level MAE ($|\text{shots}_{\text{agent}} - \text{shots}_{\text{oracle}}|$), while the red line represents the 200-episode moving average. 74
5.9 Scatter chart of agent performance with 2000 training episodes 77
5.10 Scatter chart of agent performance with 2000 training episodes 78
5.11 Scatter chart of agent performance with 2000 training episodes 79
5.12 Scatter chart of agent performance with 3000 training episodes 80
5.13 Scatter chart of agent performance with 3000 training episodes 81
5.14 Scatter chart of agent performance with 3000 training episodes 82
5.15 Scatter chart of agent performance with 5000 training episodes 83

5.16 Scatter chart of agent performance with 5000 training episodes 84
5.17 Scatter chart of agent performance with 5000 training episodes 85
5.18 Comparative scatter plot overlaying the policy of four agents trained for different durations. The 3000-episode agent (green circles) demonstrates superior alignment with the Oracle's optimal shot count. 86

Acknowledgements

Ringrazio Antonio Brogi, Alessio Bocci e Giuseppe Bisicchia per avermi guidato e supportato sin da subito, offrendomi non solo disponibilità e sostegno continuo, ma soprattutto la fiducia necessaria in un momento delicato e intenso come sono stati questi ultimi mesi. Grazie per gli spunti e per avermi concesso la possibilità di un progetto di tesi che conciliasse perfettamente la mia passione per la ricerca e la mia voglia di mettermi in gioco su un dominio così attuale quanto affascinante com'è quello del Quantum Computing.

Ringrazio tutti i professori che hanno accompagnato il mio percorso universitario, in particolare coloro che hanno saputo trasmettere con passione non solo conoscenze tecniche, ma anche il senso profondo del fare ricerca e del porsi domande.

A tutti i miei insegnanti delle scuole superiori per i valori e gli strumenti che mi hanno consegnato, fondamenta ancora solide su cui continuo a costruire.
Un ringraziamento speciale, silenzioso ma necessario, a Massimiliano, mio mentore di vita, che da ormai nove mesi sorregge il cielo con un dito. Grazie per tutte le tue lezioni, dentro e fuori le aule, hai insegnato a guardare il mondo con occhi diversi, a cercare bellezza nella precisione, umanità nella logica. Continui a vivere in noi e nelle nostre azioni, così come in ogni pagina di questa tesi, come un'eco discreta ma indelebile. Per questo, e per tutto ciò che non ho fatto in tempo a dirti, ma che continuo a raccontarti, ti ringrazio. Questo traguardo è dedicato a te.

Ringrazio la mia famiglia, tutta, per avermi sempre sostenuto e per avermi insegnato il valore del sacrificio, vivendo con l'intensità di chi ti ama ogni sfida di questo percorso.
Ringrazio i miei amici e tutti coloro che hanno incrociato la loro vita con la mia lasciandomi qualcosa di buono. Grazie per essere stati miei complici, anche in un anno intenso come quello trascorso all'estero.
Un ultimo, doveroso ringraziamento va a Rebecca. In quest'ultimo anno hai sempre trovato spazio per me, dalle notti prima degli esami alle lezioni sospese tra filosofia e informatica. Grazie per ogni risata, per ogni cena, e per questo amore sano e puro che ci trasmettiamo ogni giorno. Ti aspetto in Svizzera, o in giro per il mondo, che tanto casa è dove stiamo insieme, ti amo.

1. Introduction

1.1 Context

Quantum computing represents one of the most revolutionary paradigms in computational science, poised to fundamentally transform how we approach some of humanity's most challenging computational problems. For over half a century, the exponential growth in classical computing power, famously captured by Moore's Law, has been driven by relentless miniaturization, packing ever more transistors onto silicon chips. However, this trajectory has brought us to a critical juncture: as transistors approach atomic scales, quantum mechanical effects become unavoidable. Rather than viewing these quantum phenomena as obstacles to be suppressed at great engineering cost, quantum computing embraces them as computational resources, potentially enabling a new paradigm of miniaturization and computational power that classical physics fundamentally cannot achieve [4].

This quantum approach offers three compelling motivations for its study. First, from a purely practical standpoint, the physics of our universe provides a natural roadmap: since quantum effects inevitably emerge at nanoscales, learning to harness rather than fight them may be the only viable path for continued computational progress beyond classical limits [4]. Second, leveraging quantum mechanical principles like superposition, entanglement, and interference, enables dramatic computational advantages. Certain problems that would take classical computers millennia to solve could potentially be addressed in hours or minutes on a sufficiently large quantum computer [4]. Shor's algorithm, for instance, can factor large numbers exponentially faster than the best-known classical algorithms, threatening current cryptographic systems while simultaneously enabling quantum-secure communication protocols [18]. Grover's algorithm offers quadratic speedup for unstructured search problems, and quantum simulation promises to unlock molecular and materials science calculations that are fundamentally intractable classically. Third, from a theoretical computer science perspective, if our goal is to understand the ultimate compu-

tational power that Nature permits, and our current best description of Nature is quantum mechanics, then studying quantum computation is not merely an engineering curiosity but a fundamental necessity [4]. To truly understand the boundaries of what is computable, we must study quantum computers, not merely classical ones.

The implications of quantum advantage extend across numerous domains of critical importance. In cryptography, quantum computers threaten to break current encryption schemes while simultaneously enabling provably secure quantum communication protocols. In drug discovery and materials science, they promise to simulate molecular interactions with unprecedented accuracy, potentially accelerating the development of new pharmaceuticals and advanced materials by decades [18]. Financial institutions envision quantum algorithms optimizing complex portfolios and risk assessments in ways impossible for classical computers. Machine learning and artificial intelligence stand to benefit from quantum-enhanced optimization and pattern recognition. Perhaps most profoundly, quantum computers may help us understand quantum systems themselves, from high-temperature superconductivity to the fundamental physics of our universe, fulfilling Richard Feynman's original vision that if nature operates according to quantum rules, we should build computers that do the same [4].

Yet despite this immense promise and the significant theoretical breakthroughs achieved since the 1980s, from Deutsch's universal quantum Turing machine to Shor's and Grover's landmark algorithms, practical quantum computing remains in its nascent stages. The journey from theoretical potential to practical quantum advantage requires overcoming substantial engineering challenges, though progress is accelerating remarkably. Indeed, even as this thesis is being written, significant milestones continue to emerge: in October 2024, Google announced their *Willow quantum chip*, demonstrating *Quantum Echoes* [6], described as the first verifiable quantum algorithm providing a new benchmark for quantum advantage. This breakthrough exemplifies both the rapid advances being made and a persistent challenge that pervades the field; quantum computers must not only perform calculations beyond classical reach, but do so in ways that can be independently verified

and trusted, a requirement that places additional demands on efficient resource utilization.

More specifically, we are firmly in the Noisy Intermediate-Scale Quantum (NISQ) era, characterized by quantum processors with a limited number of qubits (typically 50 to a few hundred) that are highly susceptible to environmental noise, decoherence, and operational errors. These imperfections impose severe constraints on the depth and complexity of quantum circuits that can be executed before the delicate quantum information is lost, rendering results unreliable.

A fundamental consequence of quantum mechanics is the probabilistic nature of its outcomes. A quantum algorithm does not produce a single, deterministic answer but rather a probability distribution over a set of possible classical states. To accurately estimate this distribution, a quantum circuit must be executed repeatedly, with each execution, or "shot", providing a single sample from this distribution. By aggregating the results of thousands, or even tens of thousands, of shots, a statistical approximation of the true output can be reconstructed.

As access to quantum hardware is predominantly facilitated through cloud platforms, these shots are a finite and often costly resource, consuming both time and computational credits. The naive approach of assigning a large, fixed number of shots to every circuit is highly inefficient. Different circuits, depending on their complexity and the specific hardware backend they are run on, require vastly different numbers of shots to achieve a stable and accurate result. This inefficiency has motivated the development of adaptive strategies.

The foundational work for this thesis was made possible by one such adaptive strategy: the *Incremental Execution (IE) framework*. Developed under the guidance of my supervisors, Antonio Brogi, Alessandro Bocci, and Giuseppe Bisicchia, the IE framework operates by executing circuits in small, iterative batches. After each batch, it uses statistical distance metrics, such as the Total Variation Distance (TVD), to assess whether the accumulated probability distribution has converged to a stable state. More shots are added only if they meaningfully improve the statistical accuracy of the result. While this iterative method represents a significant improvement over static allocation, it is fundamentally based on a

set of predefined heuristics and fixed thresholds. Its greedy, step-by-step decision-making process lacks a global, long-term strategy, a limitation that directly inspires the learning-based approach explored herein.

1.2 Problem Statement

While the Incremental Execution (IE) framework provides an effective heuristic-based solution for adaptive shot allocation, its reliance on predefined rules and local, iterative convergence checks inherently limits its optimality. The core of the problem lies in the framework's design: it follows a manually-tuned, greedy strategy that may not represent the most efficient way to navigate the complex trade-offs between shot economy and statistical accuracy, especially across a diverse range of quantum circuits and hardware platforms. This limitation motivates a central research question: can a more powerful, learning-based paradigm discover a superior shot allocation policy?

This thesis frames the adaptive shot allocation task not as a convergence-detection problem to be solved with fixed rules, but as a sequential decision-making problem. The challenge is to develop an intelligent agent that can learn a dynamic, globally-aware policy from experience. Such an agent must learn to balance the competing objectives of minimizing computational cost (shots) while maximizing the accuracy of the final probability distribution, adapting its strategy to the specific characteristics of the quantum task at hand. The fundamental problem, therefore, is to design and validate a system that replaces hand-crafted heuristics with a learned, optimized strategy for quantum circuit execution.

1.3 Objectives

The primary goal of this thesis is to design, implement, and evaluate a Deep Reinforcement Learning (DRL) agent for adaptive shot allocation in quantum circuit execution, demonstrating its viability as an alternative to heuristic-based frameworks. To achieve this, the following objectives are established:

- **Formalize the problem for reinforcement learning:** To formulate the dynamic

shot allocation task as a Markov Decision Process (MDP). This includes defining a comprehensive state space that captures the execution environment, an action space that allows for dynamic control over shot allocation, and an effective reward function that guides the agent toward a balance of shot efficiency and result accuracy.

- **Design and implement a DRL agent:** To develop an intelligent agent based on a Deep Q-Network (DQN) architecture. This agent will be designed to learn a complex, non-linear policy that maps state observations to optimal shot allocation decisions, overcoming the limitations of traditional tabular methods.

- **Train the agent in a simulated quantum environment:** To train the DQN agent on a diverse dataset of quantum circuits and simulated hardware backends. The training process will enable the agent to learn a robust and generalizable policy that can adapt to varying levels of circuit complexity and noise characteristics.

- **Evaluate and validate the proposed approach:** To rigorously evaluate the performance of the trained DRL agent. This involves comparing its shot efficiency and the statistical accuracy of its results against the baseline performance of the heuristic-driven Incremental Execution framework, thereby quantifying the advantages of the learning-based approach.

1.4 Thesis structure

This thesis is structured to guide the reader from the foundational concepts of quantum computing and reinforcement learning to the specific implementation and validation of the proposed adaptive shot allocation agent. The subsequent chapters are organized as follows:

- **Chapter 2, *Background***, lays the twin theoretical pillars upon which this work is built. It begins with an introduction to the fundamental principles of Quantum Computing, including the qubit, quantum states, and the challenges of the NISQ era. It then formally defines the shot allocation problem and reviews the traditional Incremental Execution framework. The second half of the chapter transitions to the

proposed solution domain, introducing the core concepts of Reinforcement Learning, from Q-Learning to Deep Q-Networks (DQN), and concludes with a primer on the underlying neural network concepts.

- **Chapter 3, *Used Technologies***, provides a practical overview of the software stack used to implement the project. It justifies the selection of key libraries such as Python as the primary language, PyTorch for constructing the neural network, OpenAI Gym for structuring the environment, and QSimBench as the essential source of quantum circuit data and simulation.

- **Chapter 4, *Agent***, presents the core contribution of this thesis: the design and implementation of the DRL agent. This chapter offers a deep dive into the system's architecture, the agent's perception through the engineered 8-feature state space, the specifics of the Deep Q-Network architecture, and the mechanics of the learning process, including initialization, exploration strategies, and the experience replay mechanism. It concludes by detailing the complete training regimen.

- **Chapter 5, *Validation***, is dedicated to the empirical evaluation of the trained agent. It outlines the experimental framework, including the dataset, data handling strategies, and the evaluation metrics used to assess performance. This chapter presents the experimental results from different training configurations (2,000, 3,000, and 5,000 episodes) and provides a detailed discussion and critical analysis of the findings, including an examination of policy degradation over extended training periods.

- **Chapter 6, *Conclusions***, summarizes the entire work. It revisits the objectives and evaluates the extent to which they were achieved, discusses the strengths and behavioral characteristics of the final learned policy, and outlines promising directions for future research. Finally, it offers reflections on the skills and knowledge acquired throughout the project.

2. Background

2.1 Quantum Computing

For over half a century, the progress of computation has been synonymous with the relentless march of classical physics and engineering. From the Turing machine as a theoretical model to the modern silicon-based processor, our computational devices are fundamentally classical systems. They operate on the principles of deterministic logic, representing information as definite binary states, bits that are unambiguously either 0 or 1. This paradigm, propelled by the miniaturization of transistors as described by Moore's Law, has yielded unprecedented computational power. However, as we approach the atomic scale, the very laws of physics that enabled this revolution begin to pose fundamental limits. At this frontier, the bizarre yet powerful rules of quantum mechanics take over.

Quantum computation is the field that investigates the computational power and other properties of computers based on quantum-mechanical principles, offering the potential to solve problems that are intractable for classical computers. It started in the early 1980s with suggestions for analog quantum computers by Yuri Manin, Richard Feynman [9], and Paul Benioff [4], reaching more digital ground when in 1985 David Deutsch defined the universal quantum Turing machine [18]. At its core, the field of quantum computation originates from a profound insight, famously articulated by Richard Feynman: simulating a quantum system with a classical computer is extraordinarily difficult [9]. The complexity arises because describing a quantum system requires a number of parameters that grows exponentially with the number of particles. Feynman's visionary suggestion was to turn this obstacle into an opportunity: if nature is quantum, why not build our computers on the same principles? This idea marked the conceptual birth of the quantum computer, a device that harnesses quantum phenomena to process information in fundamentally new ways.

Quantum computing is therefore not merely a more powerful version of classical comput-

ing; it is a different mode of computation altogether. It combines quantum mechanics and computer science with the important objective to find quantum algorithms that are significantly faster than any classical algorithm solving the same problem. [18] It leverages principles like superposition and entanglement to perform calculations that are believed to be intractable for any classical computer, now or in the future.

2.1.1 The qubit: beyond 0 and 1

The fundamental unit of classical information is the *bit*, which can exist in one of two definite states: 0 or 1. Its quantum counterpart is the quantum bit, or *qubit*.
While both classical and quantum bits encode information into physical systems, their implementations differ fundamentally. Classical bits utilize transistors to represent binary states, whereas qubits employ quantum mechanical properties such as electron spin. An electron's spin can be measured as either up or down, corresponding to the states 0 or 1, respectively (Figure 2.1). However, unlike classical bits, qubits can also exist in a *superposition* of these states.

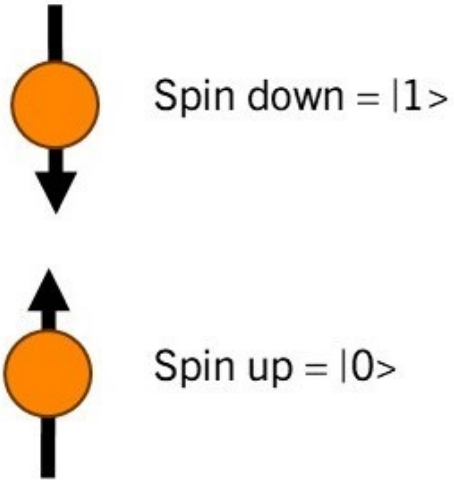

Figure 2.1: Up and down spins of an electron

The mathematical representation of a qubit employs vectors in a *Hilbert space*, which is nothing more than a vector space equipped with an inner product (or scalar product).

This formalism enables us to translate the classical bit concept $b \in \{0,1\}$ into the quantum domain by representing the binary states as two orthogonal standard basis vectors:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{2.1}$$

These vectors, denoted $|0\rangle$ and $|1\rangle$ (pronounced ”ket 0” and ”ket 1” in Dirac notation), correspond to the classical bit states and constitute the computational basis states.

The distinguishing feature of qubits lies in their ability to exist in a *superposition* of these basis states. Superposition is a fundamental quantum mechanical principle where a system exists in multiple states simultaneously until measured. Unlike classical probability, where a coin in flight is either heads or tails but we simply don’t know which, a qubit in superposition genuinely exists in both states at once [18]; a coherent quantum state where the two possibilities can interfere with each other in ways that classical probabilities cannot.

Mathematically, this superposition manifests as a linear combination of basis states, reflecting the linear structure of quantum mechanics. A pure single-qubit state is described as:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{2.2}$$

where α and β are complex numbers called *amplitudes*. These amplitudes satisfy the normalization condition $|\alpha|^2 + |\beta|^2 = 1$, which ensures $\|\,|\psi\rangle\,\| = 1$ (its length is 1). This constraint derives from the probabilistic interpretation of *quantum measurement*: $|\alpha|^2$ represents the probability of measuring the state $|0\rangle$, while $|\beta|^2$ represents the probability of measuring $|1\rangle$ [4]. The concept of *measurement* in quantum mechanics involves profound physical implications that will be explored in the subsequent section.

Geometrically, a single-qubit state can be visualized as a point on the surface of a unit sphere in three-dimensional space, known as the Bloch sphere (Figure 2.2). The north and south poles of this sphere represent the computational basis states $|0\rangle$ and $|1\rangle$, while all other surface points correspond to unique superposition states.

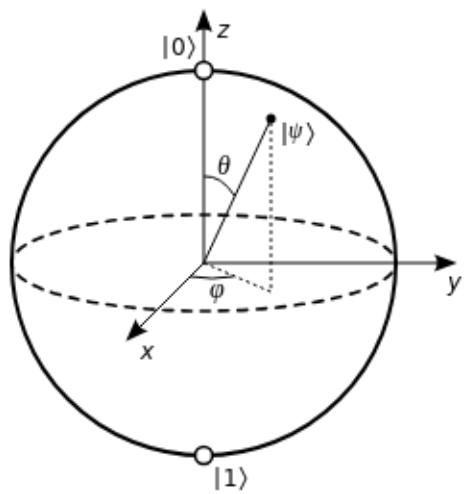

Figure 2.2: The Bloch sphere representation of a qubit state

2.1.2 Definition of a Quantum State

Having established the representation of quantum bits, we can now formally define a quantum state. Let H be a Hilbert space of dimension n. A quantum state is a unit vector $|\psi\rangle$ in this space, satisfying $\| |\psi\rangle \| = 1$. In the context of quantum computing, the Hilbert space represents the state space of a quantum register [4], with its dimension determining the number of distinct computational basis states, which translates to the number of values it can represent.

Consider the computational basis vectors $|x\rangle$ for $x \in \{0, 1, ..., n-1\}$, where each $|x\rangle$ is defined as the vector with a 1 in the x-th position and 0 elsewhere:

$$|x\rangle = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \leftarrow \text{position } x \tag{2.3}$$

These vectors form an orthonormal basis for H, satisfying $\langle x|x\rangle = 1$ and $\langle x|y\rangle = 0$ for all $x \neq y$. This set constitutes the computational basis, with each vector representing a

classical computational state.

Any quantum state can be expressed as a linear combination of these basis vectors:

$$|\psi\rangle = \sum_{x=0}^{n-1} \alpha_x |x\rangle \tag{2.4}$$

where the complex coefficients α_x must satisfy the normalization condition $\sum_{x=0}^{n-1} |\alpha_x|^2 = 1$. When multiple coefficients α_x are non-zero, the state $|\psi\rangle$ exists in a superposition of the corresponding basis states.

In the special case where $n = 2$ and $H = \text{span}\{|0\rangle, |1\rangle\}$, the Hilbert space describes a **single qubit**, the fundamental unit of quantum information.

2.1.3 Measurement: the bridge to the classical world

While a qubit can exist in an infinite continuum of superposition states, direct observation of the quantum state itself remains fundamentally impossible. The act of measurement constitutes an irreversible process that forces the qubit to *collapse* into one of the computational basis states. [18]

The concept of observation in quantum mechanics fundamentally differs from classical intuition. The famous double-slit experiment (Figure 2.3) elegantly demonstrates this principle: when electrons pass through two slits without observation, they create an interference pattern characteristic of wave behavior, indicating that each electron travels through both slits simultaneously in superposition. However, when a detector observes which slit each electron passes through, the interference pattern vanishes, and the electrons behave as classical particles. This reveals that observation itself, defined as any interaction that extracts information about the quantum state, fundamentally alters the system [4].

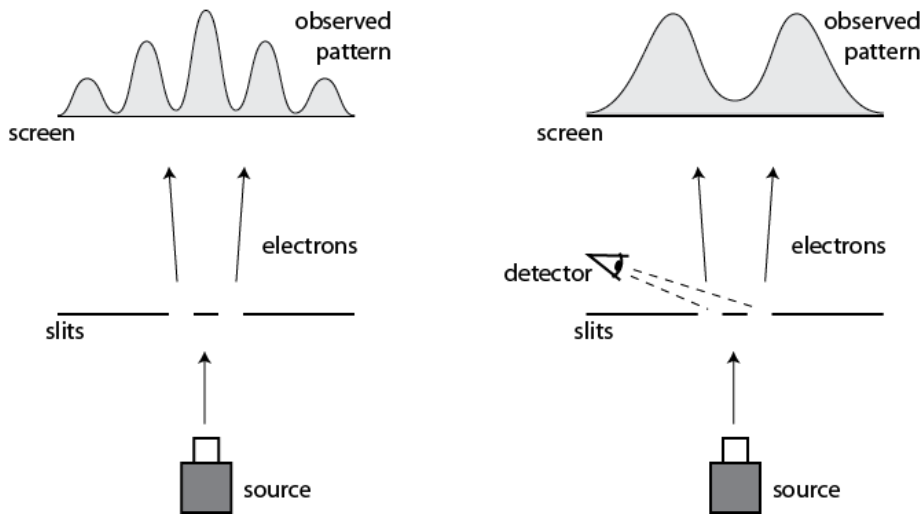

Figure 2.3: Double slit experiment, observed system vs not observed system.

In quantum computing, information extraction from quantum states occurs through specialized measurement devices integrated into the quantum processor. These devices take one or more qubits and produce classical binary values that conventional computers can process. Mathematically, measurement corresponds to a *projection operation* in linear algebra: the quantum state is projected onto one of the basis states, with the projection probability determined by the squared magnitude of the corresponding amplitude coefficient [4]. This projection transforms the quantum information into classical information that can interface with traditional computing systems.

The process of measuring a qubit is *inherently destructive*: the original superposition state cannot be recovered, and all information encoded in the complex amplitudes is irretrievably lost.

This phenomenon, often referred to as the "measurement problem", represents a fundamental constraint in quantum computation. The Stern-Gerlach experiment (Figure 2.4) provides an intuitive demonstration of this concept through sequential spin measurements [4]. Let's think of measuring atoms along a specific axis: when they are first measured along the X-axis, they split equally: 50% show positive X-spin (X^+) and 50% negative X-spin (X^-). If we then take only the X^+ atoms and measure their spin along the Z-axis, they again split equally into 50% Z^+ and 50% Z^-. Remarkably, if we take these Z^+ atoms and remeasure their X-spin, they once again split 50/50 into X^+ and X^-, the original

X-spin information has been completely erased by the intermediate Z measurement. This demonstrates that quantum measurements are fundamentally incompatible: measuring one property destroys the information about previously measured, non-commuting properties.

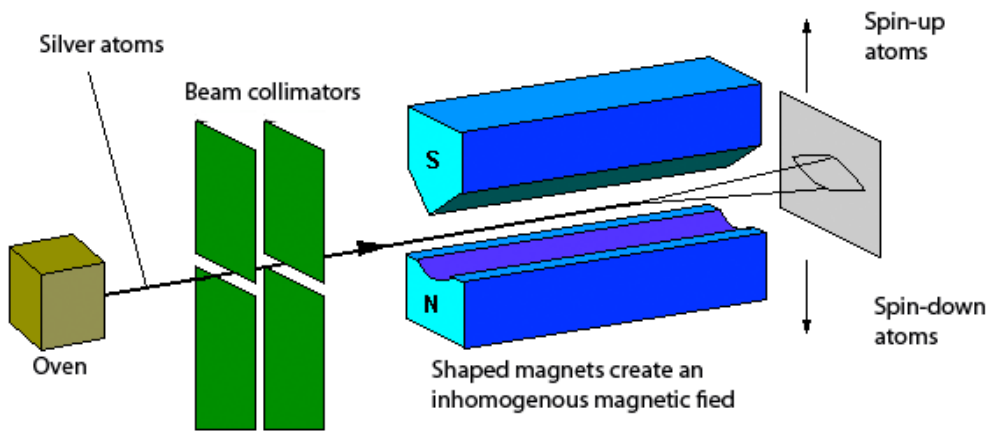

Figure 2.4: Stern-Gerlach experiment elements. The photographic plate on the right is used to observe the final beam pattern.

2.1.4 Entanglement: a deeper connection

The power of a quantum computer grows exponentially with the number of qubits. A system of n qubits has 2^n basis states (e.g., for $n = 2$, the basis states are $|00\rangle, |01\rangle, |10\rangle, |11\rangle$), and its quantum state is a superposition of all these 2^n states:

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} c_x |x\rangle \quad \text{with} \sum_x |c_x|^2 = 1 \tag{2.5}$$

This exponential state space allows for a phenomenon with no classical analogue: entanglement. An entangled state is a multi-qubit state that cannot be described as a simple product of individual qubit states. The most famous example is the Bell state, or EPR pair:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \tag{2.6}$$

In this state, the two qubits are perfectly correlated. If we give one qubit to Alice and the other to Bob and they travel to opposite ends of the galaxy, the moment Alice measures her qubit and gets the outcome 0, she *instantly* knows that a measurement of Bob's qubit will also yield 0. This correlation holds regardless of the distance separating them, an

effect Einstein famously called *spooky action at a distance* [8]. Entanglement does not allow for faster-than-light communication, but it is a critical resource that enables quantum algorithms and protocols to achieve correlations and computational feats impossible in the classical world [10].

2.1.5 Noise, decoherence, and the NISQ era

Despite their immense theoretical promise, building a practical, large-scale quantum computer is an enormous engineering challenge. The primary obstacle is *decoherence*. A quantum system is incredibly fragile; any unwanted interaction with its environment, a stray magnetic field, a temperature fluctuation, can corrupt the delicate superposition and entanglement, this process is known as decoherence, and leads to errors in the computation.

The theoretical solution to this problem is *Quantum Error Correction (QEC)* [18]. Similar to classical error correction, QEC protects information by encoding it with redundancy. For instance, a single ”logical” qubit can be encoded across multiple physical qubits [5]. By periodically measuring properties of these physical qubits (without collapsing the logical state), errors can be detected and corrected. The discovery of the *fault-tolerance threshold theorem* [18] was a landmark achievement, proving that if the error rate of individual physical gates is below a certain constant threshold, we can perform arbitrarily long quantum computations with negligible error.

However, the overhead required for fault-tolerant error correction is immense, likely requiring thousands of physical qubits to encode a single, robust logical qubit [18]. The quantum computers that exist today are not large enough or accurate enough for this. We are currently in what John Preskill has termed the *Noisy Intermediate-Scale Quantum* (NISQ) era [19]. These devices have on the order of 50 to a few hundred qubits, are too noisy to support sophisticated error correction, and their coherence times are limited, meaning they can only execute relatively “shallow” quantum circuits.

A *quantum circuit* represents the fundamental computational model for quantum algorithms, analogous to logic circuits in classical computing. It consists of a sequence of *quantum gates*, unitary operations that manipulate qubit states, applied to a register of

qubits over discrete time steps. Each gate performs a specific transformation on one or more qubits, and the gates are arranged in layers that execute sequentially. The *depth* of a quantum circuit refers to the number of sequential gate layers that must be executed, where gates acting on independent qubits can be performed in parallel within the same layer [5].

In the context of NISQ devices, ”shallow” circuits are those with limited depth, typically only dozens to hundreds of gate layers, as opposed to the millions of operations that fault-tolerant quantum computers could theoretically execute. This constraint arises from decoherence: quantum states gradually lose their coherence due to environmental interference, effectively imposing a time limit on quantum computations. Before decoherence destroys the quantum information, NISQ devices can only perform a limited number of sequential operations, restricting us to algorithms that can produce useful results within these shallow circuit depths. This fundamental limitation shapes the entire landscape of near-term quantum algorithm development.

Currently, there is no universally adopted standard for implementing qubits. Each available technology presents its own advantages and limitations in terms of performance. Table 2.1 provides a comparison of some of the most prominent approaches.

Technology	**Examples**	**Advantages**	**Disadvantages**
Superconductors [11]	IBM, Google	Fast, scalable	Sensitive to cold, noise
Ions [3]	IonQ, Honeywell	High fidelity, stability	Slow operations
Photons [21]	PsiQuantum, Xanadu	Scalable, no cooling	Emerging technology
Neutral atoms [25]	PASQAL, QuEra	Long coherence times	Cooling required
Silicon [7]	Intel	Existing infrastructure	Still in early phase
Topological [13] [1]	Microsoft (in development)	Theoretically very stable	Not yet demonstrated

Table 2.1: Quantum computing most popular technology platforms

2.2 The shot allocation problem in Quantum Computing

One of the core challenges in quantum computing arises from the probabilistic nature of quantum measurements. When a quantum circuit is executed, the measurement of qubits causes quantum states to collapse into classical outcomes according to the probability distribution defined by the quantum computation. To estimate this probability distribution accurately, quantum circuits must be executed multiple times (with each execution being called a ”shot”), and the measurement outcomes aggregated statistically.

For instance, consider Grover’s algorithm searching for a marked item in a database of four elements. After applying the appropriate number of Grover iterations, the quantum state vector might be:

$$|\psi\rangle = 0.1\,|00\rangle + 0.1\,|01\rangle + 0.97\,|10\rangle + 0.1\,|11\rangle \tag{2.7}$$

where $|10\rangle$ is the marked item. While the state vector indicates a $|0.97|^2 \approx 94\%$ probability of measuring the correct answer, a single shot would yield only one of the four possible outcomes. Running 1,000 shots might produce approximately 940 measurements of $|10\rangle$, 20 of $|00\rangle$, 20 of $|01\rangle$, and 20 of $|11\rangle$, empirically reconstructing the underlying probability distribution that the quantum algorithm encoded in the state vector.

This requirement for multiple shots introduces a critical resource optimization problem. Running too few shots results in poor statistical accuracy and unreliable results, while running excessive shots wastes computational resources, time, and financial costs on quantum cloud platforms. The challenge becomes more complex because different quantum circuits require vastly different numbers of executions to reach stable results, where ”stable” means the estimated probability distribution has converged to within an acceptable statistical error margin. Some circuits may achieve this statistical convergence with hundreds of shots, while others require tens of thousands.

Traditional shot allocation methods typically rely on heuristic estimates based on circuit characteristics or developer experience. Researchers and practitioners often make educated guesses about the required number of shots, usually erring on the side of excess to ensure sufficient statistical accuracy. These approaches are generally not adaptive, they

determine the shot count before execution and cannot adjust based on the actual convergence behavior observed during runtime. This reliance on human expertise and conservative overestimation, while ensuring reliable results, leads to systematic inefficiency in quantum computing workflows, as many circuits receive far more shots than necessary for the desired accuracy level.

2.2.1 Incremental Execution: a convergence-based approach

This thesis work was in first place made possible thanks to the development of an adaptive and iterative strategy called the *Incremental Execution Framework* [2]. To address the inefficiencies of the fixed-shot, "one-size-fits-all" approach, the Incremental Execution Framework is designed to reduce the total shot usage by dynamically determining the required number of shots based on the observed convergence of the circuit's output distribution.

The core principle is adaptive convergence detection. Instead of committing to a large, predetermined shot count, the framework executes the quantum circuit in small batches (e.g., 50 shots per iteration) and continuously monitors the stability of the accumulated probability distribution. The framework employs statistical distance metrics, such as the *Total Variation Distance* (TVD), to quantitatively assess whether the probability distribution has converged to a stable state. This iterative refinement allows the algorithm to allocate more shots only when they meaningfully improve the statistical accuracy of the result. *"In other words, we only keep adding more shots as long as they meaningfully improve our estimate of the outcome distribution. Once changes become negligible, we terminate"* [2].

Convergence vs. Error Minimisation

The Incremental Execution Framework adopts a philosophical approach to shot allocation that distinguishes between two fundamentally different objectives: minimizing estimation error and detecting distributional convergence.

The presence of noise on contemporary quantum hardware creates a fundamental distinction between theoretical error models and practical system behavior. Unlike in ideal

simulations where estimation error can be arbitrarily reduced, the performance of physical quantum processors is bounded by the effects of decoherence, gate infidelity, and readout noise. These factors establish a practical limit on the fidelity of any estimated distribution. Within this framework, the objective is not the indefinite minimization of error with respect to an ideal *ground truth*, the theoretical, noise-free probability distribution that would result from executing the quantum circuit on perfect hardware, but rather the detection of **distributional convergence**. This stance recognizes that, in practice, convergence is achieved when the outcome distribution stabilizes, meaning that subsequent shots no longer induce significant statistical changes. This approach focuses on the efficient acquisition of a statistically stable result rather than the pursuit of an unattainable, noise-free ideal.

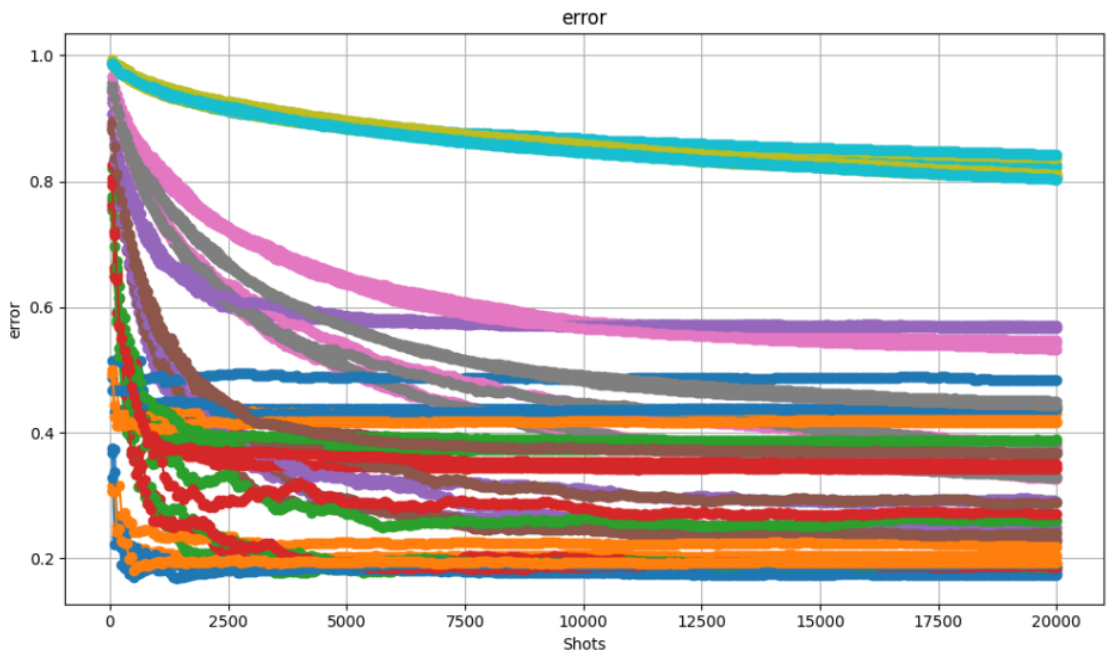

Figure 2.5: Error increasing the number of shots. [2]

Algorithmic protocol

The incremental execution algorithm follows a well-defined iterative protocol. The process can be summarized in the following high-level steps:

1. **Initialization:** The process begins by establishing an initial, small batch size for the

first execution of the quantum circuit.

2. **Execute and Accumulate:** The circuit is run for the specified number of shots, and the measurement outcomes are aggregated into a cumulative probability distribution.

3. **Convergence Evaluation (Stopping Criterion):** The current cumulative distribution is compared to a previous state of the distribution (e.g., the distribution from the previous iteration) to assess whether stability has been reached. This evaluation returns a boolean value indicating convergence.

4. **Stability Confirmation:** To prevent premature termination due to statistical fluctuations, the algorithm checks if the stability condition has been met for a predetermined number of consecutive iterations, denoted by k.

5. **Termination or Continuation:** If k stable iterations are achieved or a maximum shot budget is reached, the process terminates. Otherwise, a new batch size is calculated, and the loop continues from Step 2.

This adaptive loop ensures that simple circuits that converge quickly consume fewer resources, while more complex circuits are allocated a sufficient number of shots to achieve statistical reliability.

Key Criteria for Convergence

The flexibility of the incremental execution framework is rooted in its three main decision-making components: the stopping criterion, the stability criterion, and the next shots criterion.

Stopping Criterion This function assesses if the probability distribution has sufficiently converged in the current iteration. Several implementation methods can be used:

- **Distance Threshold:** The most direct approach, where the TVD between the current and a previous cumulative distribution must fall below a predetermined threshold (e.g., 0.01).

- **Moving Average:** To mitigate the effects of statistical noise, this method evaluates whether the average TVD over the last several iterations remains below the threshold.
- **Quorum-Based:** This method requires a majority ("quorum") of recent stability checks within a sliding window to satisfy the threshold, providing a more robust measure of convergence.

Stability Criterion A single low-distance measurement could be a statistical fluctuation rather than true convergence. This criterion enhances the algorithm's robustness by requiring the stopping criterion to be met for k consecutive iterations before terminating.

Next Shots Criterion When convergence is not achieved, this rule governs the size of the subsequent batch. Strategies can be either constant (e.g., always adding 50 more shots) or adaptive (dynamically adjusting the batch size based on the rate of convergence).

Mathematical foundations and formal definitions

At the heart of convergence detection lies a distance metric between probability distributions. The primary metric used in this framework is the **Total Variation Distance (TVD)**, defined for two discrete probability distributions P and Q over a set of outcomes X as:

$$\mathrm{TVD}(P,Q) = \frac{1}{2}\sum_{x\in X} |P(x) - Q(x)| \tag{2.8}$$

A small TVD value indicates that the two distributions are close. While TVD is a robust choice, other metrics such as Kullback-Leibler (KL) Divergence or Hellinger Distance could also be employed.

The stopping criteria are formally defined based on the history of measurements. Let $h = [r_1, r_2, \ldots, r_t]$ be the history of frequency count snapshots at each iteration t. Let $R_t = \sum_{i=1}^{t} r_i$ be the cumulative frequency count, and let $p_t = R_t/N_t$ be the normalized empirical distribution at iteration t, where N_t is the total number of shots so far. The stopping criteria can be formalized as follows:

- **Delta Distance Criterion:** Given a threshold $\delta > 0$, the criterion is met if the distance to a previous distribution $p_{t'}$ is small:

$$\text{Delta}(p_t, p_{t'}) = \begin{cases} \text{True} & \text{if TVD}(p_t, p_{t'}) < \delta \\ \text{False} & \text{otherwise} \end{cases} \tag{2.9}$$

The convergence behavior is illustrated in Figure 2.6, which shows the TVD between consecutive cumulative distributions for various circuit-backend pairs. The distance rapidly decreases and then stabilizes, indicating convergence.

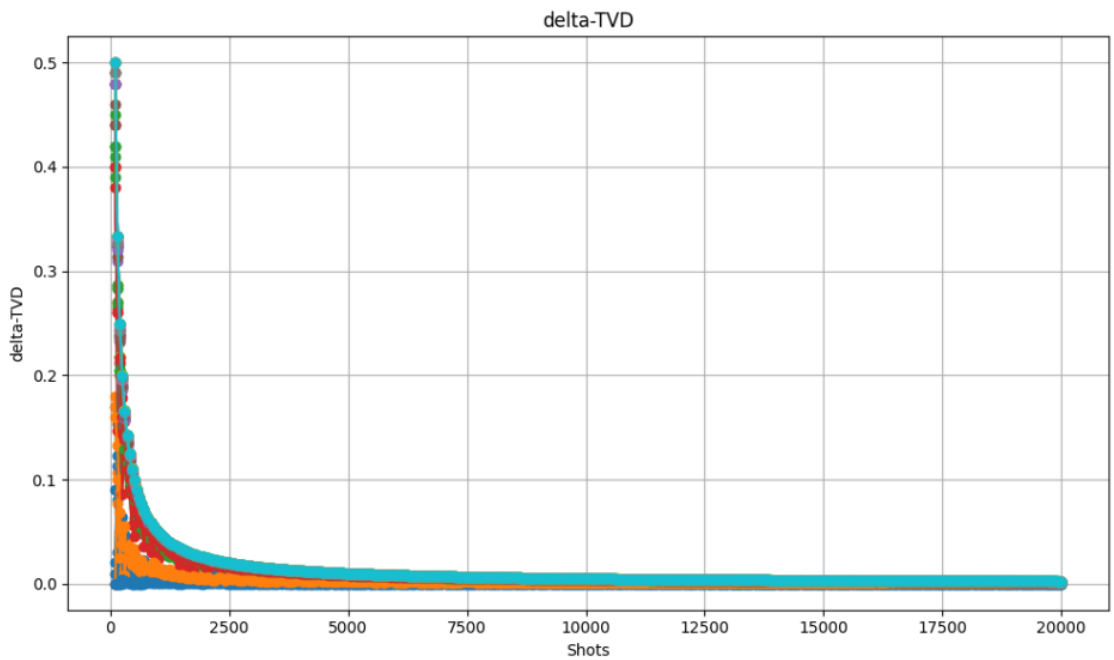

Figure 2.6: The Total Variation Distance (TVD) between consecutive cumulative distributions as a function of the number of shots. Each line represents a different circuit-backend pair. The distance decreases sharply at first and then plateaus, signaling that further shots provide diminishing returns. [2]

Performance trade-offs and limitations

The incremental execution framework introduces a fundamental trade-off between shot economy and final accuracy. Stricter convergence criteria (e.g., a lower TVD threshold or a higher k value for stability) yield superior accuracy but consume more shots. Conversely,

relaxed thresholds lead to greater shot savings at the potential expense of precision.

However, a more fundamental limitation lies in the framework's architectural design itself. The Incremental Execution strategy, with its specific batch sizes, convergence metrics, and stopping criteria, represents only *one* of countless possible approaches to the shot allocation problem. The choice of TVD as a distance metric, the decision to use fixed batch sizes, and the particular convergence detection algorithm are all design decisions made by the researchers. A different research team might develop an entirely different framework with alternative metrics, adaptive batch sizing, or different convergence philosophies, potentially achieving different trade-offs between accuracy and efficiency.

This observation reveals an important limitation: the incremental execution framework embeds human assumptions and design choices about *how* the shot allocation problem should be solved. While these choices are well-motivated and empirically effective, they inherently constrain the solution space to the specific strategy envisioned by its designers. This raises a natural question: could there exist superior strategies that we have not yet conceived?

This limitation motivates the exploration of a reinforcement learning approach, which will be discussed in the following section. Rather than constraining the solution to a predefined algorithmic structure, an RL agent can potentially discover novel shot allocation strategies by learning directly from the optimization objective, without being biased by human preconceptions of how the problem should be solved.

2.3 Reinforcement Learning agents

Traditional rule-based approaches to optimization problems require researchers to manually define criteria and make design choices upfront. This limitation has led many fields to explore more adaptive, data-driven methods that can learn from experience. One of the most powerful paradigms for addressing sequential decision-making problems is **Reinforcement Learning (RL)**, a branch of machine learning that enables *agents* to learn optimal strategies through interaction with their environment, rather than following explicitly programmed rules.

Unlike traditional algorithmic approaches that embed specific assumptions about how a problem should be solved, reinforcement learning agents discover solutions by exploring the problem space and learning from the consequences of their actions. This fundamental difference makes RL particularly well-suited for problems where the optimal strategy is unknown, difficult to specify manually, or may vary significantly across different problem instances.

2.3.1 The principles of Reinforcement Learning

Reinforcement Learning is a paradigm of machine learning concerned with goal-directed learning from interactions. It constitutes one of the three fundamental categories of machine learning (Figure 2.7), and addresses the problem of how an intelligent **agent** should take **actions** in an **environment** to maximize a cumulative numerical reward signal [22]. Unlike supervised learning, the agent is not told which actions to take; instead, it must discover which actions yield the most reward by trying them out. This trial-and-error search and the focus on delayed rewards are the two most important distinguishing features of reinforcement learning.

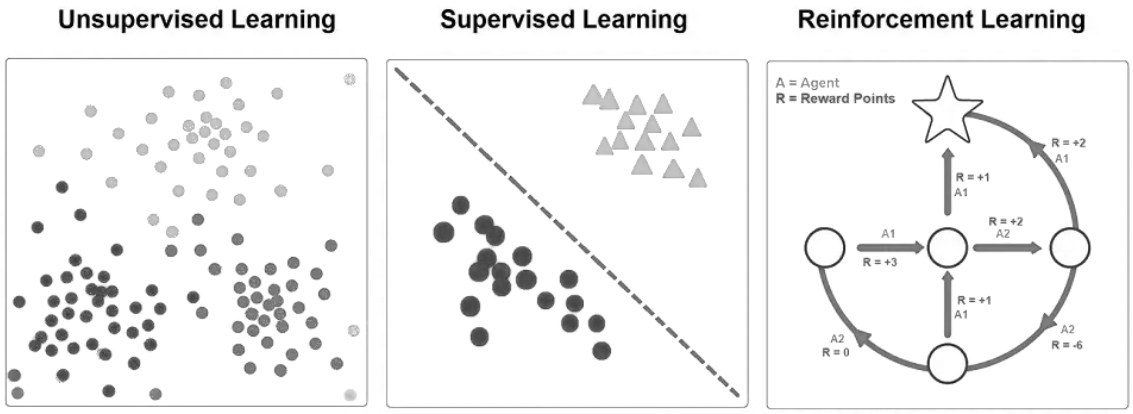

Figure 2.7: Machine Learning three broad categories [15]

The RL framework is formalized through the concept of a **Markov Decision Process (MDP)**, which models the interaction between the agent and its environment. This interaction forms a feedback loop, as illustrated in Figure 2.8.

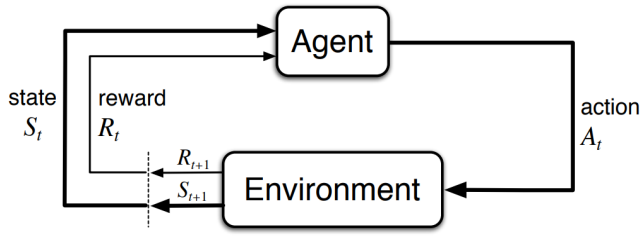

Figure 2.8: The basic feedback loop in Reinforcement Learning. At each time step, the agent receives the environment's state, selects an action, and the environment responds with a new state and a reward. [22].

The core components of this loop are:

- **Agent**: The learner and decision-maker. Its role is to learn a strategy for acting.
- **Environment**: Everything outside the agent. It receives the agent's actions and responds by presenting a new state and a reward.
- **State** (S_t): A representation of the environment at time step t. It contains all the information the agent needs to make a decision. For example, in a walking robot problem, the state could include the angles of every joint, the robot's velocity, and data from its camera [15].
- **Action** (A_t): One of the choices available to the agent. The action modifies the state of the environment. For the walking robot, an action could be the specific combination of torques applied to its motors.
- **Reward** (R_{t+1}): A scalar numerical value that the environment sends to the agent. The reward provides immediate feedback on the desirability of the agent's action in the preceding state.

The agent's sole objective is to maximize the total reward it accumulates over the long run. This process of learning from interaction is what distinguishes RL from other machine learning paradigms like supervised and unsupervised learning, which typically operate on static datasets [22].

2.3.2 Maximizing cumulative reward

The immediate reward signal indicates what is good in the short term, but the agent's goal is to maximize the cumulative reward over the long term. This is formalized as maximizing the expected **return**. The return, G_t, is the sum of future rewards, but often with a discount factor, $\gamma \in [0,1]$:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.10}$$

The discount factor determines the present value of future rewards. A reward received k steps in the future is only worth γ^{k-1} times what it would be if received immediately. This encourages the agent to be a little more short-sighted, which is advantageous for two main reasons: future rewards may be less reliable or uncertain, and a reward received sooner is often preferable to one received later [22].

2.3.3 Policies and value functions

An agent's behavior is defined by its **policy**, denoted $\pi(a|s)$, which is a mapping from states to probabilities of selecting each possible action. The learning process is about finding an optimal policy, π_*, that maximizes the expected return.
To evaluate how good a state or a state-action pair is, reinforcement learning uses **value functions**.

- The **state-value function**, $v_\pi(s)$, is the expected return when starting in state s and following policy π thereafter.
- The **action-value function**, $q_\pi(s,a)$, is the expected return when starting in state s, taking action a, and then following policy π.

The distinction between immediate reward and long-term value is crucial. An action might yield a small immediate reward but lead to states from which much larger rewards can be obtained. The value function captures this long-term potential and is therefore the primary guide for making optimal decisions [22].

2.3.4 Exploration vs. exploitation

A core challenge in RL is the trade-off between **exploration** and **exploitation**. To maximize its reward, the agent must exploit actions that it has learned are effective. However, to discover such actions, it must explore by trying actions it has not selected before. The agent cannot pursue both exclusively; it must explore to make better action selections in the future, but it must also exploit its current knowledge to obtain reward [22]. This dilemma is central to RL, as the agent's own actions determine the data it collects for learning.

2.3.5 Q-Learning

One of the most foundational algorithms in reinforcement learning is **Q-learning**, a model-free approach that trains an agent to assign optimal values to its possible actions based on its current state [23]. Being model-free means the agent does not need to understand the environment's underlying dynamics (i.e., transition probabilities or reward functions); instead, it learns directly from the experience it gathers through trial and error. For any finite Markov Decision Process, Q-learning can find an **optimal policy**, which maximizes the expected total reward over all successive steps, given sufficient exploration [16].

The Q-Table: A Tabular Approach

For environments with a finite and discrete number of states and actions, the Q-learning algorithm can be implemented using a simple lookup table, known as a **Q-table**. This table stores a value for every possible state-action pair, representing the "quality" or expected long-term reward of taking a specific action in a specific state. As shown in Figure 2.9, the agent's policy is to consult this table and select the action with the highest Q-value for its current state.

-1 →	-1	-1	$\cdots$
-1	-10	+10	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\ddots$

Q	a1	a2	a3	a4
S1	Q_{11}	Q_{12}	Q_{13}	Q_{14}
S2	Q_{21}	Q_{22}	Q_{23}	Q_{24}
S3	Q_{31}	Q_{32}	Q_{33}	Q_{34}

Figure 2.9: Gridworld environment example (left) and corresponding Q-table mapping states to actions (right). In this example the agent's policy is to select the action with the highest Q-value starting from the top-left cell(indicated by the arrow in the left table).

The agent learns the values in this table iteratively. After taking an action A_t in state S_t and observing the reward R_{t+1} and the new state S_{t+1}, the corresponding Q-value is updated using the **Bellman equation**:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right] \tag{2.11}$$

Here, α is the learning rate, which controls how much the new information overrides the old estimate. The term inside the brackets is the *temporal-difference (TD) error*, representing the difference between the new, improved estimate of the Q-value (the immediate reward plus the discounted value of the best action in the next state) and the old one. By repeatedly applying this update, the Q-values gradually converge towards the optimal action-value function, $q_*(s,a)$.

The need for function approximation

While the tabular approach is intuitive, its primary drawback is a lack of scalability. As the state and action spaces expand, the memory required to store a complete Q-table grows exponentially. For any problem with a large or continuous state space—such as controlling a robot with many sensors or navigating a complex environment—it is computationally intractable to populate a discrete table for every possible state. This problem is commonly known as the **curse of dimensionality**.

To overcome this limitation, it is more effective to express the tabular policy as a parameterized **function** that approximates the Q-value. As illustrated in Figure 2.10, this function takes the state and action as input and returns a Q-value that represents the quality of that action.

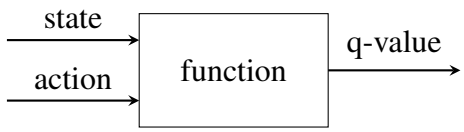

Figure 2.10: To handle large state spaces, the discrete Q-table is replaced by a function that maps a state-action pair to a Q-value.

This shift from a table to a function approximator is the key concept that enables reinforcement learning to be applied to complex, high-dimensional problems and provides the foundation for more advanced methods like Deep Q-Networks.

2.3.6 Deep Q-Learning (DQN)

The tabular approach of Q-learning fails when the state or action spaces become very large or continuous, a problem known as the *curse of dimensionality*. It is impossible to populate a table for every conceivable state.

To solve this, we can use an approximation function, such as a neural network, to estimate the Q-value function. This is the core idea behind **Deep Q-Learning** [15].

A **Deep Q-Network (DQN)** is a neural network that takes the state s as input and outputs the Q-values for all possible actions in that state, $Q(s, \cdot; \theta)$, where θ are the weights of the network, as shown in Figure 2.11.

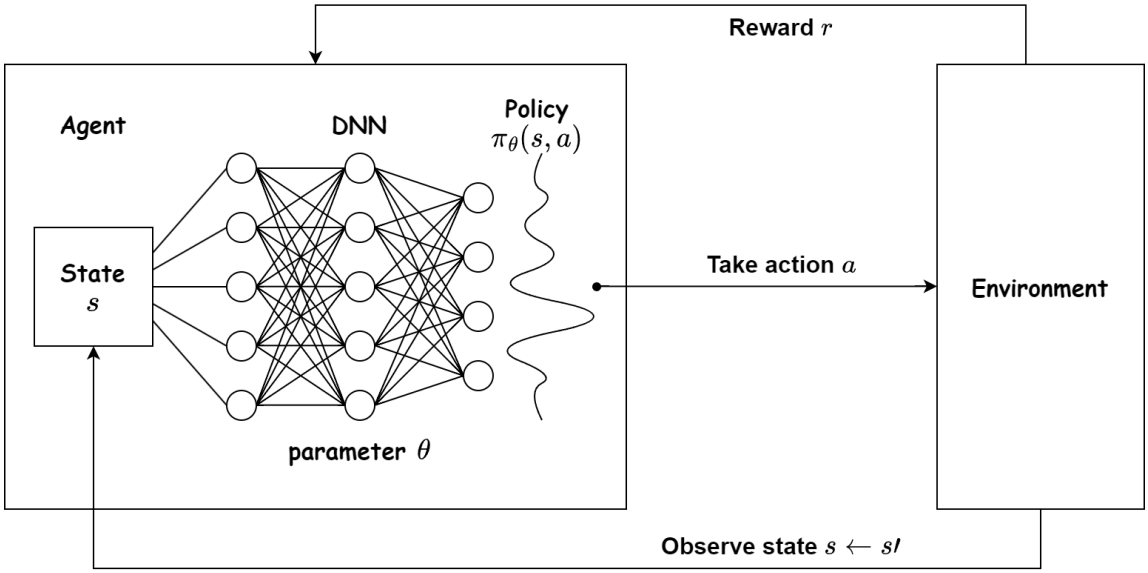

Figure 2.11: The architecture of a Deep Q-Network (DQN). The neural network acts as a function approximator, taking the state as input and outputting the Q-values for each possible action. [24]

Since training neural networks with the highly correlated data from RL can be unstable, DQN introduced two key innovations to address this problem [14, 17]:

- **Experience Replay:** The agent's experiences $(S_t, A_t, R_{t+1}, S_{t+1})$ are stored in a large replay memory buffer (Figure 2.12). During training, the network is updated using mini-batches of randomly sampled experiences from this buffer. This breaks the strong temporal correlations in the sequence of observations and smooths the learning process.

- **Target Network:** DQN uses a second, separate neural network whose weights are kept fixed for a period of time (Figure 2.12). This target network is used to generate the target Q-values for the Bellman update. By providing a stable target, it prevents the oscillations that can occur when a single network is updated to chase its own rapidly changing predictions. In other terms, while the main Q-Network is randomly initialized and represents the "current knowledge", we also use a target network with the same architecture that represents the "stable reference". The goal is to compare

them, compute a loss, and use this loss to update the parameters of the Q-Network through backpropagation, iterating to improve them continuously.

These techniques allow RL agents to learn effective policies in high-dimensional state spaces.

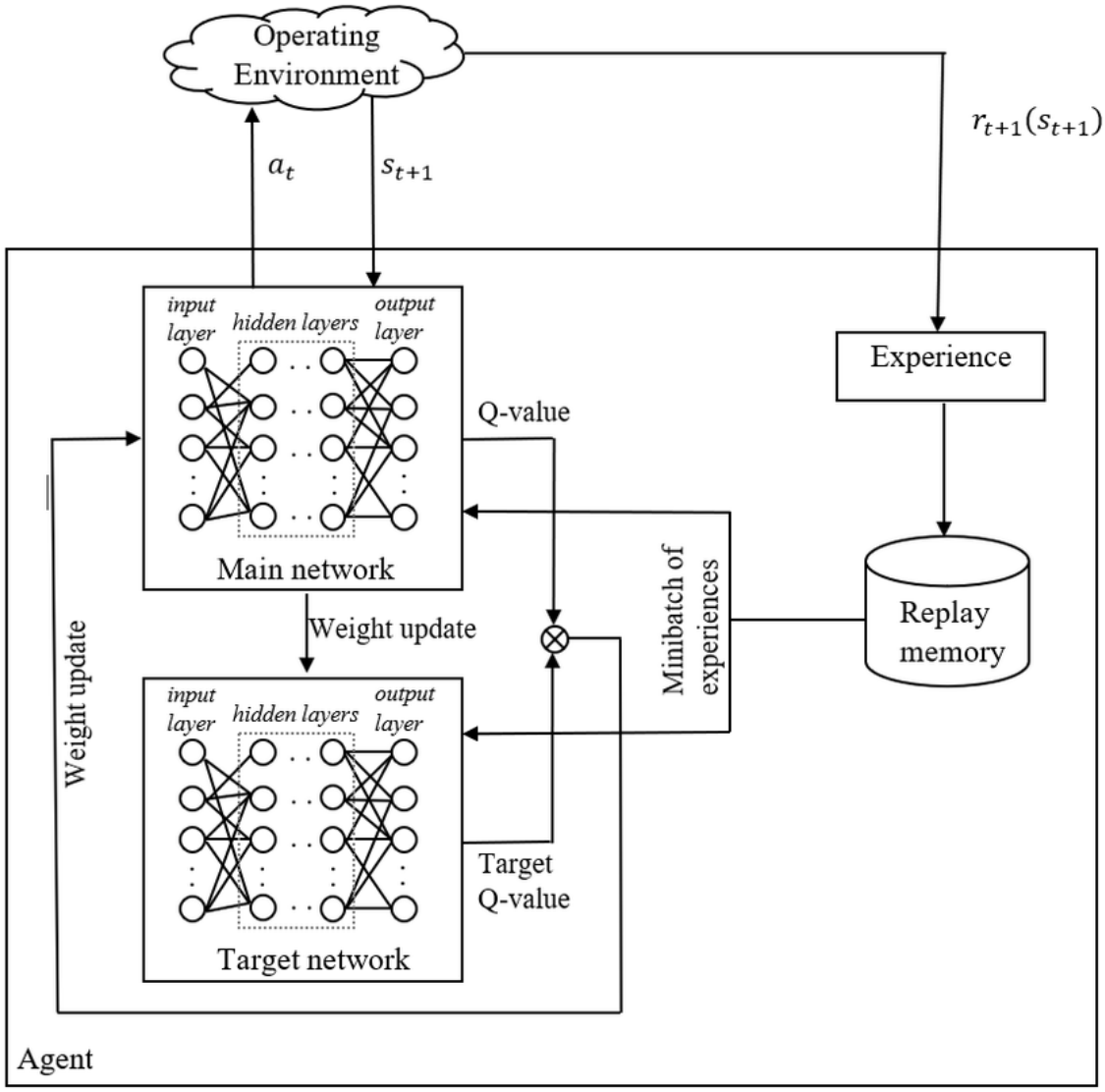

Figure 2.12: DQN architecture with its main network, the target network, and replay memory [20].

2.4 Understanding Neural Networks

As we have established, the tabular approach to Q-learning becomes computationally infeasible in large or continuous state spaces. Deep Q-Learning (DQN) overcomes this by employing a neural network as a powerful function approximator. Before detailing the specific architecture of our agent in Chapter 4, it is useful to first build a conceptual understanding of what a neural network is and how it learns.

While a comprehensive treatment of neural network theory is beyond the scope of this work, we can effectively treat the network as a "*black box*" learner. This black box takes a set of inputs and, through a learnable internal structure, produces a set of outputs. The magic of the network lies in its ability to automatically adjust its internal configuration to learn the complex, non-linear relationships between inputs and desired outputs without being explicitly programmed.

To understand how this is achieved, we can briefly examine the fundamental components and the learning process.

2.4.1 Architecture: layers and neurons

Neural networks are organized into sequential *layers* of computational units called *neurons* (or nodes). The most basic architecture, which we employ in our DQN, is the **feed-forward neural network** or **multi-layer perceptron (MLP)**, where information flows unidirectionally from input to output without loops.

A typical MLP consists of three types of layers:

1. **Input layer**: receives the raw input features.
2. **Hidden layers**: one or more intermediate layers that transform the input into increasingly abstract representations. Our DQN architecture uses three hidden layers with 512, 256, and 128 neurons respectively.

3. **Output layer**: produces the final prediction. For our DQN, this is a 2-dimensional vector containing Q-values for the CONTINUE and STOP actions.

Each neuron in a layer performs a simple computation: it takes a weighted sum of its inputs, adds a bias term, and applies a non-linear **activation function**. Mathematically, for a neuron j in layer l:

$$a_j^{(l)} = f\left(\sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)}\right) \tag{2.12}$$

where $w_{ij}^{(l)}$ are the **weights** connecting neuron i from the previous layer to neuron j, $b_j^{(l)}$ is the **bias**, $a_i^{(l-1)}$ are the activations from the previous layer, and $f(\cdot)$ is the activation function.

2.4.2 Weights and biases: the learnable parameters

The weights and biases are the **parameters** that the network learns during training. Initially set to small random values, these parameters are iteratively adjusted to minimize the difference between the network's predictions and the desired outputs. For a network with millions of parameters (as is common in deep learning), this parameter space is vast, yet gradient-based optimization methods can efficiently navigate it.

2.4.3 Activation Functions: Introducing Non-linearity

Without activation functions, stacking multiple layers would be pointless, the composition of linear functions is itself linear, offering no advantage over a single layer. Activation functions introduce the **non-linearity** that enables neural networks to learn complex patterns and make the *Universal Approximation Theorem* possible.

> *A feedforward neural network with just one hidden layer can approximate any continuous function to any desired accuracy given enough neurons.*

Our DQN primarily uses the **Rectified Linear Unit (ReLU)** activation function:

$$\mathrm{ReLU}(x) = \max(0, x) \tag{2.13}$$

ReLU has become the standard choice for hidden layers due to its computational simplicity and effectiveness in mitigating the vanishing gradient problem during training. The output layer typically uses a **linear activation** (no transformation) when predicting continuous values like Q-values, or a **softmax** function when producing probability distributions.

2.4.4 Forward Propagation: Making Predictions

When the network receives an input, it performs **forward propagation**: the input flows through each layer sequentially, with each neuron computing its activation based on the previous layer's outputs, until the final prediction emerges from the output layer. This is the "inference" mode, using the network's current parameters to make predictions.

2.4.5 Learning: backpropagation and gradient descent

The true power of neural networks lies in their ability to learn from data. This learning happens through an algorithm called *backpropagation* combined with *gradient descent*.

The process works as follows:

1. **Forward pass**: The network makes a prediction for a given input.

2. **Loss computation**: A *loss function* (or error function) measures how far the prediction is from the desired output. For regression tasks like Q-value prediction, mean squared error (MSE) is commonly used:

$$\mathscr{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{2.14}$$

   where y_i is the true value (target) and $\hat{y}_i$ is the network's prediction.

3. **Backward pass (backpropagation)**: This is where the "magic" happens. Using the chain rule from calculus, backpropagation efficiently computes the **gradient** of the loss with respect to every weight and bias in the network, essentially determining how much each parameter contributed to the error.

4. **Parameter update (gradient descent)**: The parameters are adjusted in the direction that reduces the loss. The update rule for a weight w is:

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial \mathscr{L}}{\partial w} \tag{2.15}$$

   where α is the **learning rate**, a hyperparameter controlling the size of the update steps.

This four-step cycle repeats thousands or millions of times across many training examples, gradually sculpting the parameter space until the network learns to make accurate predictions.

2.4.6 The black box revisited

While we have now seen inside the black box, the key takeaway is this: from a practical standpoint, we can treat the neural network as a powerful, differentiable function that automatically learns complex input-output mappings when given:

1. A dataset of input-output pairs (or in reinforcement learning, experiences)
2. A loss function defining what "good performance" means
3. Sufficient training time and computational resources

This black box perspective allows us to focus on the higher-level problem of designing effective reinforcement learning systems, which is the true contribution of this thesis.
The neural network is simply our tool, a sophisticated function approximator that makes Deep Q-Learning possible by enabling us to handle the high-dimensional, continuous state spaces that would be impossible to address with traditional tabular methods.
In the following chapter, we leverage this understanding to construct our DQN agent, where the neural network serves as the core mechanism for learning optimal shot allocation policies in quantum circuit execution.

3. Used Technologies

The implementation of the Deep Reinforcement Learning (DRL) agent and the simulated quantum environment that we will see in the next chapters relies on the Python programming language and its extensive ecosystem of scientific computing and machine learning libraries. This chapter quickly outlines the core technologies selected for the development of this thesis project.

Programming language: Python

The entire framework is implemented in **Python**. While often praised for its simplicity, the choice of Python for this project was driven by a more fundamental constraint: it is currently the only language that serves as the *lingua franca* for both the Quantum Computing and the Machine Learning research communities.

For a project of this nature—where an RL agent must iteratively interact with a quantum simulator—seamless integration is paramount. The domain-specific library used for the thesis, `QSimBench`, is native to Python, as are the vast majority of modern quantum frameworks (e.g., Qiskit, Cirq). Simultaneously, the leading deep learning frameworks are Python-first. Attempting to bridge a C++ or Julia machine learning backend with a Python quantum simulator would have introduced significant latency and architectural complexity. Python allowed for the creation of a unified memory space where the Agent (the learner) and the Environment (the quantum simulator) could exchange high-dimensional data vectors efficiently.

Deep Learning framework: PyTorch

To implement the Deep Q-Network (DQN) agent, this thesis utilizes **PyTorch** (`torch`), one of the leading open-source machine learning libraries. PyTorch provides the fundamental building blocks for the neural network architecture and the learning mechanisms.

In the context of this work, PyTorch is used for:

- **Network Construction:** The `torch.nn` module is used to define the Multi-Layer Perceptron (MLP) architecture, including linear layers, activation functions (ReLU), and dropout regularization.
- **Optimization:** The `torch.optim` module provides the Adam optimizer, which updates the network weights during the backpropagation phase to minimize the temporal difference error.
- **Tensor Operations:** PyTorch manages the efficient manipulation of high-dimensional tensors representing state vectors and Q-values.
- **Hardware Acceleration:** The library supports CUDA (Compute Unified Device Architecture), enabling the offloading of tensor computations to NVIDIA GPUs. This capability allows for significantly faster training times by parallelizing the forward and backward passes of the neural network, which is particularly beneficial when training over thousands of episodes.

Reinforcement Learning interface: OpenAI Gym

The interaction between the agent and the quantum simulation is structured using the **OpenAI Gym** toolkit (now maintained as Gymnasium). This library provides a standardized interface for reinforcement learning environments.

By inheriting from the `gym.Env` class, the `IterativeQuantumEnv` encapsulates the quantum circuit execution logic into a formal Markov Decision Process (MDP). It defines the `action_space` (Discrete) and the `observation_space` (Box), and implements the standard `reset()` and `step()` methods, ensuring the agent can interact with the problem space in a consistent, framework-agnostic manner.

Quantum simulation: QSimBench

The source of quantum circuit data and execution simulation is **QSimBench** [**?**]. This library serves as the backbone of the training environment, providing a reproducible bench-

mark suite of quantum circuits.

The project utilizes QSimBench to:

- Access a standardized dataset of quantum algorithms (e.g., Grover, QAOA) across varying qubit sizes.
- Simulate realistic hardware noise profiles (e.g., `fake_kyiv`) to ensure the agent learns to navigate real-world imperfections.
- Generate shot-level measurement outcomes via the `get_outcomes` function, which are aggregated by the environment to construct the dynamic state features.

Scientific computing and visualization

Several auxiliary libraries are essential for data manipulation and result analysis:

- **NumPy:** The fundamental package for scientific computing in Python. It is used extensively for handling numerical arrays, performing state vector normalization, and computing statistical metrics such as the Shannon entropy and variance of the probability distributions.
- **Matplotlib:** Used to generate the evaluation dashboard presented in the Validation chapter. It visualizes the training progression, reward history, and error distributions, enabling the qualitative analysis of the agent's policy.
- **TQDM:** Provides progress bars for the training loops and evaluation processes, offering real-time monitoring of the experiment's status.

4. Agent

In the previous chapters, we established the theoretical foundations of this work, examining the probabilistic nature of quantum measurement and the resulting shot allocation problem. We discussed the Incremental Execution (IE) framework and its limitations, and introduced Reinforcement Learning (RL) as a powerful approach to sequential decision-making in high-dimensional spaces.

This chapter transitions from theory to practice. Here, we detail the concrete implementation of the Deep Reinforcement Learning (DRL) agent designed to solve the adaptive shot allocation problem. The core idea is to move away from the predefined rules of the IE framework and instead empower an agent to learn its own, potentially superior, strategy for determining when a quantum circuit's output has converged. The agent achieves this by observing thousands of simulated quantum circuit executions and receiving feedback, via a reward signal, on the quality of its stopping decisions.

The central question we address is practical: *How can we construct an intelligent agent that learns to efficiently and accurately allocate shots for quantum circuit execution?* To answer this, we will dissect the entire system, beginning with a high-level overview to understand its components and their interactions, and then attempting to open the "*black box*" of the agent itself to examine its neural network architecture and decision-making logic. Finally, we will describe the training process in detail, the data the agent learns from, the reward function that guides its behavior, and the mechanisms that make the learning process stable and effective.

Why Deep Reinforcement Learning

The selection of Deep Reinforcement Learning (DRL) as the methodological framework for this problem was a deliberate design choice, made after considering and rejecting other machine learning paradigms. The unique characteristics of the adaptive shot allo-

cation task, its sequential nature, the absence of step-by-step labels, and its goal-oriented objective, make Deep Reinforcement Learning a uniquely suitable approach.

One might first consider framing the problem as a standard supervised regression task, where a model would be trained to predict the optimal total shot count based on the initial static features of a circuit (its algorithm, size, and backend). However, this static, one-shot prediction model was deemed unsuitable because it is fundamentally blind to the dynamic information that unfolds during the execution itself. Such a model would make its prediction at the outset and could not adapt if the circuit converged faster or slower than expected. It would be entirely ignorant of the real-time signals from the "instrument panel", such as the stabilizing entropy or the diminishing rate of change.

An unsupervised learning approach was also considered. Techniques like clustering could identify groups of circuits with similar convergence trajectories, and anomaly detection could flag when the distribution's rate of change becomes statistically insignificant. While powerful for pattern recognition, such methods are fundamentally *descriptive*, not *prescriptive*. An unsupervised model could highlight that a distribution has stabilized, but it offers no inherent mechanism to translate that observation into a goal-oriented action. A separate layer of heuristics would still be needed to decide whether to stop. In contrast, Deep Reinforcement Learning (DRL) is inherently goal-directed. The reward function drives the agent to not only recognize patterns but to directly map those patterns to the actions that will maximize its long-term objective. It learns not just what the state is, but what it should do about it. Therefore, Deep Reinforcement Learning (DRL) was chosen because it is the only paradigm that natively addresses all three critical aspects of the problem: it is sequential and adaptive, it can learn from delayed outcomes without step-by-step supervision, and it is inherently optimized to take actions that achieve a specific, quantifiable goal.

4.1 System architecture overview

To train an agent capable of making good decisions, we must first construct a complete learning ecosystem, as explained in Chapter 2.3.1. This system is composed of three primary, interacting components:

- **The quantum environment:** Serves as the "*playground*" for the agent. It simulates the iterative execution of quantum circuits, providing the agent with observations about the current state of the execution and allowing it to take actions. It is built upon the `qsimbench` library, which provides a standardized and diverse set of quantum circuits and realistic noise models.

- **The oracle:** Acts as the "teacher" or ground truth. It is a direct implementation of the Incremental Execution (IE) framework, as detailed in Section 2.2.1. For any given quantum circuit, the Oracle determines the optimal number of shots required to reach convergence according to its TVD-based heuristic rules. Its answer is used exclusively to score the agent's performance at the end of an episode and is never shown to the agent during its decision-making process.

- **The agent:** This is the learner and decision-maker, implemented as a Deep Q-Network (DQN). At each step, it observes the state provided by the environment and chooses one of two actions: CONTINUE executing more shots or STOP the execution. Its goal is to learn a policy that maximizes the rewards it receives over time.

The interaction between these components forms a closed loop, as shown in Figure 4.1. An episode begins with the environment selecting a quantum circuit problem. At each step, the environment presents the agent with a state vector describing the execution. The agent takes an action, and the environment transitions to a new state. This continues until the agent decides to stop. At that point, the environment consults the Oracle to determine the optimal shot count, calculates a final reward based on the agent's performance, and sends this reward back to the agent, completing the feedback loop and providing a powerful learning signal.

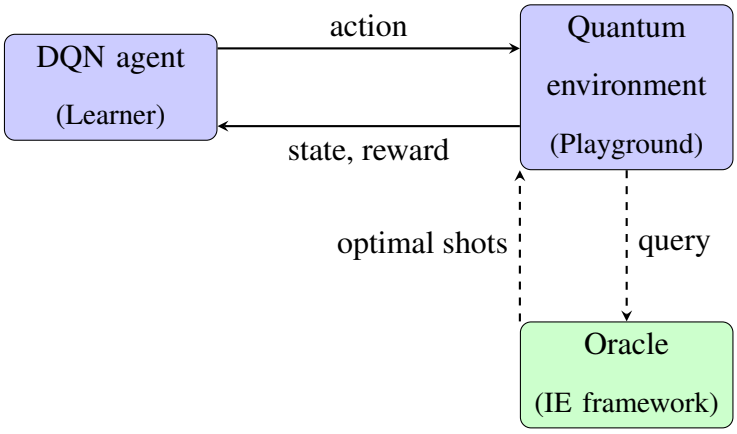

Figure 4.1: High-level architecture of the reinforcement learning system for adaptive shot allocation. The agent observes states and takes actions iteratively until it decides to stop. Only at episode termination the environment consults the Oracle to evaluate the agent's performance and compute the final reward. The Oracle's optimal shot count is never revealed to the agent during its decision-making process.

4.2 Diving in agent's perception: the state space

As we established in the background, an agent's ability to make intelligent decisions is entirely dependent on the information it receives. An agent that is "blind" to the dynamics of quantum circuit convergence cannot learn an effective policy. To overcome this, the state vector was provided with an 8-feature vector that functions as an "*instrument panel*" providing real-time information.

The state $\mathbf{s}_t$ at any time step t is represented as an 8-dimensional vector, with each feature normalized to the range $[0, 1]$ to facilitate neural network training. These features fall into two categories: **static features** that characterize the circuit and hardware configuration, and **dynamic features** that evolve as more shots are executed and provide real-time feedback about convergence progress.

During the development of this agent, I experimented with two different state representations to understand the impact of dynamic information on learning performance. The

initial version operated with a limited 4-feature state vector containing only static information about the circuit and backend. While this simpler representation allowed the agent to learn basic patterns, I hypothesized that it might be making the agent somewhat "blind" to the real-time dynamics of quantum circuit convergence. To test this, I augmented the state vector with four additional dynamic features that function as an "instrument panel", providing the agent with the temporal information needed to learn a more robust and nuanced policy. The comparative performance of these two approaches is analyzed in detail in Chapter 5.

Static features

These features describe the problem itself and do not change during an episode. They allow the agent to identify the type of circuit it is dealing with and are defined in the `_get_state` function of the `IterativeQuantumEnv` class.

1. **Algorithm encoding:** A normalized value representing the quantum algorithm (e.g., QAOA, Grover). This helps the agent learn that different algorithms may have different convergence patterns.

```
# Each algorithm (e.g., 'grover', 'qaoa', 'vqe') is assigned a unique integer index during initialization via self.alg_map (e.g., {'grover': 0, 'qaoa': 1})
# This index is retrieved: self.alg_map.get(alg, 0)
# Normalized to [0, 1] by dividing by (total_algorithms - 1)
# Example: If there are 3 algorithms, indices 0,1,2 become 0.0, 0.5, 1.0

alg_norm = self.alg_map.get(alg, 0) / (len(self.alg_map) - 1) if len(self.alg_map) > 1 else 0.5
```

Code 4.1: Algorithm normalization feature construction

2. **Problem size:** The number of qubits, normalized. This is a crucial indicator of complexity.

```
# The size represents the number of qubits in the quantum circuit
# Normalized by dividing by an assumed maximum size of 15 qubits (based on typical values in the qsimbench dataset)
#   - 3-qubit circuit: 3/15 = 0.2
#   - 10-qubit circuit: 10/15 = 0.67

size_norm = size / 15.0
```

Code 4.2: Circiut size feature construction

3. **Backend encoding:** A normalized value representing the simulated hardware. This allows the agent to learn policies that are sensitive to different noise models.

```
# Each quantum backend (e.g., 'ibmq_lima', 'ionq_harmony') is assigned a unique integer index via self.backend_map during initialization
# This index is retrieved: self.backend_map.get(backend, 0)
# Normalized to [0, 1] by dividing by (total_backends - 1)

backend_norm = self.backend_map.get(backend, 0) / (len(self.backend_map) - 1) if len(self.backend_map) > 1 else 0.5
```

Code 4.3: Backend type feature construction

4. **Current shots:** The number of shots executed so far, normalized by the maximum budget. This serves as a basic progress bar.

```
# Directly from self.current_shots (accumulated across all batches)
# Normalized by dividing by self.max_shots (typically 20,000)
# 1000 shots used: 1000/20000 = 0.05 (5% of budget)
# 15000 shots used: 15000/20000 = 0.75 (75% of budget, time to consider stopping)
```

```

shots_norm = self.current_shots / self.max_shots
```

Code 4.4: Current shot count feature construction

Dynamic features: the "instrument panel"

These features are updated at every step and give the agent real-time insight into the statistical behavior of the measurement outcomes.

5. **Distribution entropy:** it is computed using Shannon entropy, which measures the uncertainty or "*spread*" of a probability distribution. This feature is vital for distinguishing between algorithm types based on their output. For example, a successful Grover's search should result in a low-entropy, "peaked" distribution concentrated on the correct answer. In contrast, an algorithm exploring a large state space, like QFT [12], might produce a high-entropy, "flat" distribution. This allows the agent to learn what a "converged" state looks like for different kinds of problems.

   The normalized Shannon entropy, H_{norm} is calculated as follows. First, for each unique measurement outcome x from the set of all possible outcomes $\mathscr{X}$, we estimate its probability $p(x)$ from the measurement counts:

   $$p(x) = \frac{\text{count}(x)}{N} \quad \text{where} \quad N = \sum_{i \in \mathscr{X}} \text{count}(i) \tag{4.1}$$

   Here, $\text{count}(x)$ is the number of times outcome x was measured, and N is the total number of shots so far.

   Next, the Shannon entropy $H(\mathscr{X})$ in bits is computed using the standard formula:

   $$H(\mathscr{X}) = -\sum_{x \in \mathscr{X}} p(x) \log_2 p(x) \tag{4.2}$$

   Finally, the entropy is normalized to a value between $[0, 1]$ by dividing by a predefined maximum entropy value based on our dataset.

   $$H_{\text{norm}} = \frac{H(\mathscr{X})}{H_{\text{max}}} \tag{4.3}$$

```
# Convert counts to probabilities: p_i = count_i / total_counts
# Example: {'00': 0.55, '01': 0.22, '10': 0.13, '11': 0.10}
# Apply Shannon entropy formula
# Normalize by max entropy

entropy = self._compute_distribution_entropy(
    cumulative_outcomes)
```

Code 4.5: Shannon entropy feature construction

6. **Distribution variance:** this is the statistical variance of the outcome probabilities themselves. It provides a complementary view to entropy. While entropy measures overall spread, variance reinforces the agent's perception of the distribution's shape. It helps the agent differentiate between a distribution with one major peak (high variance) and one with several smaller peaks (lower variance), which might require different stopping criteria.

```
# Convert counts to probabilities: [p, p, ..., p]
# Calculate variance
# Apply scaling factor and clip to [0, 1]
# Peaked distribution {'0000': 0.9, others: 0.002 each} -> high
    variance
# Flat distribution {'00': 0.25, '01': 0.25, ...} -> low
    variance

variance = self._compute_distribution_variance(
    cumulative_outcomes)
```

Code 4.6: Distribution variance feature construction

7. **Rate of change (via TVD):** it is the Total Variation Distance between the cumulative distribution of all prior batches and the distribution of only the most recent batch. This directly informs the agent's core task. A high value means the distribution is still changing rapidly and convergence is far. A low value signals that the distri-

bution is stabilizing and convergence is near. The agent can learn the fundamental rule: "stop when the change is small".

```
# Take the last batch of outcomes from self.outcome_history[-1]
# Take all previous batches and aggregate them
# Normalize both into probability distributions
# Calculate Total Variation Distance (TVD) between them
# Intuition is: if adding 50 more shots changes the
    distribution dramatically, then keep going. If adding 50
    more shots barely changes anything, it's time to stop

rate_of_change = self._compute_rate_of_change()
```

Code 4.7: Rate of change feature construction

8. **Relative progress:** it was introduced to solve the problem of temporal ambiguity, providing the agent with a normalized measure of execution completeness. It is computed by dividing the number of shots executed so far by a heuristic estimate of the total shots typically required for a problem of a given size.

   The heuristic estimate itself is calculated using the formula: $\text{typical_shots} = \text{size} \times 500$. This simple linear model is based on the empirical observation that larger circuits generally require more shots to converge. The scaling factor of 500 was chosen as a practical approximation that provides a reasonable timeline across both easy and hard problems, preventing the agent from becoming either too impatient with complex circuits or too hesitant with simple ones.

   Its purpose is to enable the agent to develop time-dependent policies by providing crucial context for "interpreting" dynamic signals like the Rate of Change. Without this context, it's harder for the agent to distinguish between two scenarios:

   (a) **Early, transient stability:** A low Rate of Change observed early in an execution (when *Relative Progress* is low) is likely to be statistical noise from an insufficient sample size. Here, the correct policy is to continue.

   (b) **Late, genuine convergence:** The same low Rate of Change observed late in

an execution (when *Relative Progress* is high) is a strong signal that the distribution has genuinely stabilized. Here, the correct policy is to stop.

By conditioning its decisions on both the immediate stability (Rate of Change) and its position in the expected timeline (Relative Progress), the agent learns to differentiate between easy problems that converge quickly and hard problems that require a longer initial "*burn-in*" period to achieve a reliable statistical estimate.

```
# Estimate typical shots needed based on problem size
typical_shots = size * 500 # Heuristic: larger circuits
    generally need more shots

# Compute ratio: current_shots / typical_shots
# Clip to maximum of 1.0
relative_progress = min(self.current_shots / typical_shots,
    1.0)
```

Code 4.8: Relative progress feature construction

4.3 The agent's brain: Deep Q-Network architecture

As introduced in Section 2.3.6, the vast and continuous nature of the state space makes a traditional Q-table computationally infeasible. Our agent therefore employs a Deep Q-Network (DQN), which uses a neural network as a powerful function approximator to estimate the action-value function, $Q(s,a)$. The specific neural network architecture chosen for this task is a *multi-layer perceptron* (MLP). An MLP is a foundational type of *feedforward neural network*, consisting of an input layer, one or more hidden layers of neurons, and an output layer. In this structure, information flows *unidirectionally* from the input state vector through the hidden layers, which learn to recognize and abstract complex patterns in the data, to the final output layer that produces the Q-values. This layered approach makes the MLP particularly effective at modeling the non-linear relationships required to solve our shot allocation problem.

4.3.1 Network Structure

Its architecture is as follows:

- **Input Layer:** 8 neurons, corresponding to the 8 features of the state vector.
- **Hidden Layer 1:** 512 neurons with a ReLU activation function. This large initial layer allows the network to create rich, high-dimensional combinations of the input features. Consider that this network's depth and width were chosen to match the complexity of the 8-feature state. For the previous, simpler 4-feature vector, a less expressive architecture with roughly half the neurons per layer would have provided adequate representational capacity
- **Hidden Layer 2:** 256 neurons with a ReLU activation function.
- **Hidden Layer 3:** 128 neurons with a ReLU activation function. This deeper architecture provides the capacity to learn more complex and non-linear relationships between the state features and the optimal action.
- **Output Layer:** 2 neurons, with a linear activation. Each neuron corresponds to an action, and its output value is the predicted Q-value for that action ($Q(s, \text{CONTINUE})$ and $Q(s, \text{STOP})$).

To make a decision, the agent simply feeds the current state vector into the network and selects the action corresponding to the neuron with the highest output value. Figure 4.2 shows this specific Deep-Q Network scheme.

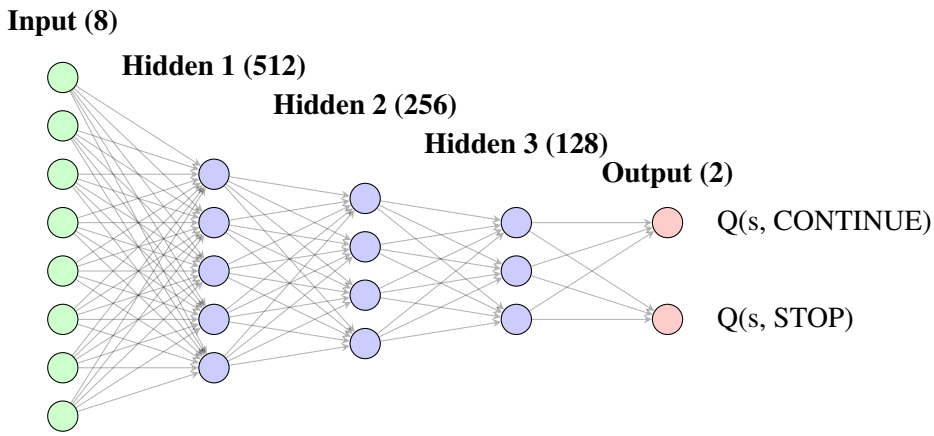

Figure 4.2: Detailed neuron-level view of the DQN architecture. Only representative neurons are shown for each layer; actual layer sizes are indicated in parentheses.

The network employs a funnel architecture where layer widths decrease geometrically ($512 \rightarrow 256 \rightarrow 128$), encouraging hierarchical feature learning. Early layers learn to detect basic patterns in the state features, such as recognizing low TVD values or high shot counts, while deeper layers combine these into higher-level concepts like "the circuit has converged" or "more exploration is needed". Code 4.9 shows how the Deep Q-Network (DQN) class was implemented using `pytorch`.

```
class DQN(nn.Module):
    """
    DQN with 8 input features and 2 output neurons
    """
    def __init__(self, input_size: int, output_size: int):
        super().__init__()
        # Definition of layers
        self.net = nn.Sequential(
            nn.Linear(input_size, 512), nn.ReLU(),
            nn.Dropout(0.1),  # Light regularization
            nn.Linear(512, 256), nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, output_size)
```

```
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

Code 4.9: Deep Q-Network class

To prevent overfitting during training, I incorporated *dropout layers* with a 10% dropout rate after the first two hidden layers. Dropout randomly deactivates neurons during training, forcing the network to learn more robust representations that don't rely too heavily on any single feature. This regularization technique is particularly important given that our training dataset, while diverse, is still finite.

4.4 Defining the learning process

The `Agent` class represents the core of the Deep Q-Network implementation, orchestrating the entire reinforcement learning process through two fundamental mechanisms: experience replay and periodic target network updates. This architecture addresses the inherent instability that arises when training neural networks with highly correlated sequential data, a challenge that is particularly acute in reinforcement learning contexts.

4.4.1 Architecture and initialization

The agent maintains two identical neural networks with distinct roles in the learning process. The primary Q-network (`q_net`) serves as the agent's decision-making engine, continuously updated to reflect the most recent learning experiences. In parallel, the target network (`target_net`) provides stable Q-value estimates during training, preventing the oscillations that would occur if a single network attempted to chase its own rapidly evolving predictions.

```
def __init__(self, state_size: int, action_size: int,
             learning_rate: float = 1e-4, gamma: float = 0.99):
    # The main network for decision-making
```

```
    self.q_net = DQN(state_size, action_size)
    # The target network for stable Q-value estimation
    self.target_net = DQN(state_size, action_size)
    self.update_target()  # Initialize with identical weights

    # Experience replay buffer
    self.memory = deque(maxlen=200000)
    self.optimizer = optim.Adam(self.q_net.parameters(), lr=
   learning_rate)
```

Code 4.10: Agent initialization with dual networks

The agent also initializes a replay memory buffer with a capacity of 200,000 experiences, implemented as a double-ended queue for efficient memory management. This buffer stores the agent's interaction history as tuples of (state, action, reward, next_state, done), enabling the crucial experience replay mechanism.

The use of a deque with a maximum length of 200,000 for the experience buffer is a deliberate design choice. As new experiences are added beyond this capacity, the oldest experiences are automatically discarded, ensuring the buffer contains recent, relevant data while maintaining a fixed memory footprint. This capacity was chosen to be large enough to provide diverse training batches while remaining computationally manageable.

4.4.2 Action selection and exploration

The agent employs an ε-greedy policy for action selection, balancing exploration of new strategies with exploitation of learned knowledge:

```
def act(self, state: np.ndarray) -> int:
    # Exploration: random action with probability epsilon
    if random.random() < self.epsilon:
        return random.randint(0, self.action_size - 1)

    # Exploitation: select action with highest Q-value
    state_tensor = torch.FloatTensor(state).unsqueeze(0)
```

```
    with torch.no_grad():
        q_values = self.q_net(state_tensor)

    # Return the action with the highest Q-value
    return torch.argmax(q_values).item()
```

Code 4.11: Epsilon-greedy action selection

The agent's exploration rate ε starts at 1.0 and gradually decays to a minimum of 0.05. Initially, with ε at its maximum, the agent's actions are purely random. This ensures it gathers a wide variety of experiences across the problem space before it has developed a reliable policy. As training progresses, ε is systematically reduced, shifting the agent's behavior from broad exploration toward the exploitation of its learned knowledge. The final 5% exploration rate is maintained to prevent the policy from stagnating, allowing the agent to occasionally discover better strategies even in later stages of training.

4.4.3 Experience replay mechanism and Bellman update with target networks

Experience replay addresses a fundamental challenge in reinforcement learning: the strong temporal correlations in sequential experiences. Rather than learning from experiences in the order they occur, the agent stores them in memory and samples random mini-batches for training:

```
def replay(self, batch_size: int = 64):
    if len(self.memory) < batch_size:
        return  # Wait for sufficient experiences

    # Random sampling breaks temporal correlations
    minibatch = random.sample(self.memory, batch_size)
    states, actions, rewards, next_states, dones = zip(*minibatch)

    # Convert to tensors for neural network processing
    states = torch.FloatTensor(np.array(states))
```

```
    actions = torch.LongTensor(actions)
    rewards = torch.FloatTensor(rewards)
    next_states = torch.FloatTensor(np.array(next_states))
    dones = torch.FloatTensor(dones)
```

Code 4.12: Experience replay implementation in replay function

This random sampling strategy serves multiple purposes: it breaks the correlation between consecutive samples that could destabilize gradient descent, enables more efficient use of collected experiences by allowing each transition to be used multiple times, and smooths the learning process by averaging updates over diverse experiences.

The `replay` function also contains the core of the learning process, lying in the application of the Bellman equation, which relates the value of a state-action pair to immediate rewards and future values. The agent computes current Q-values using the main network and target Q-values using the frozen target network:

```
    # Current Q-values from main network
    current_q_values = self.q_net(states).gather(1,
                            actions.unsqueeze(1)).squeeze(1)

    # Target Q-values using target network (stable reference)
    with torch.no_grad():
        next_q_values = self.target_net(next_states).max(1)[0]
        # Bellman equation: Q(s,a) = r + gamma * max_a' Q(s',a')
        target_q_values = rewards + (1 - dones) * self.gamma *
next_q_values

    # MSE loss between predicted and target Q-values
    loss = F.mse_loss(current_q_values, target_q_values)

    # Standard PyTorch training loop: zero gradients, backpropagate,
update weights
    self.optimizer.zero_grad()
    loss.backward()
```

```
    self.optimizer.step()

    # Decay epsilon (reduce exploration over time)
    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay
```

Code 4.13: Bellman equation implementation with target network in replay function

The term `(1 - dones)` elegantly handles terminal states: when an episode ends (`done = True`), future rewards become zero, as no further actions are possible. This ensures that the Q-value for terminal states reflects only the immediate reward.

4.4.4 Network synchronization

Periodically, the target network must be synchronized with the evolving Q-network to incorporate learned improvements while maintaining stability:

```
def update_target(self):
    """Copies weights from main network to target network"""
    self.target_net.load_state_dict(self.q_net.state_dict())
```

Code 4.14: Target network update

This synchronization typically occurs every few hundred steps, striking a balance between stability (keeping the target fixed) and incorporating new learning (updating with improved weights). The specific update frequency becomes a hyperparameter that can significantly impact learning stability and convergence speed.

Through this carefully orchestrated interplay of experience replay and target networks, the agent learns to navigate the complex state space of quantum circuit execution, discovering optimal shot allocation policies that balance computational efficiency with result accuracy. The combination of these techniques enables stable learning in the high-dimensional, continuous state space characteristic of the quantum shot allocation problem.

4.5 The training regimen

This section details the data, reward structure, and procedural steps of the training process that enables the Deep Reinforcement Learning (DRL) agent to learn its adaptive shot allocation policy. The entire regimen is designed to teach the agent to solve a sequential decision-making problem: at each step of a quantum circuit's execution, it must decide whether to continue investing resources (shots) or to stop.

4.5.1 Training Data and Environment Setup

The foundation of any machine learning system is its data. For this task, the agent learns from simulated quantum circuit executions provided by the `qsimbench` library. This library offers a standardized and diverse set of problems, ensuring the agent is exposed to a wide variety of conditions. As shown in the library's index, the dataset includes multiple quantum algorithms (e.g., `dj`, `ghz`, `qaoa`), circuit sizes, and simulated backends that emulate the noise profiles of real quantum hardware (e.g., `fake_fez`, `fake_kyiv`).

The training "*playground*" is encapsulated in the `IterativeQuantumEnv` class, which simulates the iterative execution process. A key enhancement in our training regimen is addressing the natural imbalance in the dataset, where "easy" problems (requiring few shots) are far more common than "hard" ones. To ensure the agent properly learns to handle these difficult cases, we implement a difficulty-biased sampling strategy. Upon initialization, the environment categorizes all problems based on their oracle-defined optimal shot count and, during training, oversamples from the "hard" category with a 70% probability, as shown in Code 4.15.

```
def _categorize_by_difficulty(self, difficulty_threshold=5000):
    """
    Categorizes all problems into "easy" and "hard" based on their
   optimal shot count.
    This allows us to bias our sampling toward harder problems
```

```
during training.
    """
    self.easy_triplets = []
    self.hard_triplets = []

    for triplet in self.all_triplets:
        optimal = find_optimal_shots(*triplet)
        if optimal > difficulty_threshold:
            self.hard_triplets.append(triplet)
        else:
            self.easy_triplets.append(triplet)

    print(f"Categorized {len(self.hard_triplets)} hard problems (>{difficulty_threshold} shots)")
    print(f"Categorized {len(self.easy_triplets)} easy problems (<={difficulty_threshold} shots)")
```

Code 4.15: Difficulty-biased sampling setup within the environment's initialization. The agent is deliberately exposed to more challenging problems to improve its robustness.

4.5.2 The reward structure

The reward function serves as the fundamental learning signal that shapes the agent's decision-making policy throughout training. The reward structure employs a two-choices design that combines continuous, step-wise feedback with a dominant terminal signal to guide the agent toward statistically reliable and resource-efficient quantum circuit execution strategies.

Step-wise efficiency penalty

During each decision cycle in which the agent selects the 'CONTINUE' action (action 0), a small but persistent penalty of `-0.02` is applied. This negative reward serves as a continuous pressure mechanism that discourages unnecessarily prolonged episodes. By accumulating across multiple time steps, this penalty creates an implicit incentive for the

agent to minimize the total number of shots executed, thereby promoting computational efficiency and resource conservation. Importantly, this penalty is deliberately kept small to ensure it does not overwhelm the more significant terminal reward signal.

Terminal reward signal

The primary learning signal is delivered as a large-magnitude terminal reward, computed and assigned only at the conclusion of an episode when the agent executes the 'STOP' action (action 1) or reaches the maximum shot limit. This terminal reward is calculated in the `_terminate` method (Code 4.16) and uses different reward magnitudes for different error types, reflecting that some mistakes are more costly than others in quantum computing.

The asymmetric structure distinguishes between two fundamentally different failure modes:

- **Undershooting (premature termination):** When the agent terminates execution before accumulating the oracle's predetermined optimal shot count ($\texttt{error} < 0$), it receives a substantial negative reward calculated as:

$$R_{\text{undershoot}} = -1.0 - \frac{|\text{error}|}{\text{optimal_shots}} \tag{4.4}$$

  This formulation treats undershooting as a catastrophic failure warranting severe punishment. The base penalty of -1.0 is augmented by a normalized deficit term that scales proportionally with the magnitude of the shortfall relative to the required shots. This design reflects the critical importance of statistical reliability in quantum measurements: insufficient sampling leads to high-variance estimates and scientifically invalid results, making this the most undesirable outcome in the problem domain.

- **Overshooting (on-time or delayed termination):** When the agent stops at precisely the optimal point ($\texttt{error} = 0$) or continues beyond it ($\texttt{error} > 0$), it receives a positive reward indicating successful achievement of statistical convergence. The reward follows an exponential decay function:

$$R_{\text{overshoot}} = e^{-0.0005 \times \text{error}} \tag{4.5}$$

This reward structure achieves maximum value ($R = 1.0$) for perfect stopping decisions where `error` equals zero. As the agent overshoots and wastes computational resources, the reward decays exponentially with a decay rate of 0.0005 per excess shot. This exponential decay maintains positive rewards for all overshooting scenarios (thereby distinguishing them categorically from undershooting), while simultaneously creating a smooth gradient that incentivizes precision. The relatively gentle decay rate reflects the asymmetric cost structure of the problem: overshooting wastes resources but maintains statistical validity, whereas undershooting compromises the fundamental reliability of the quantum measurement process.

This carefully calibrated asymmetric reward function encodes the core trade-off in adaptive quantum circuit execution: the agent must learn to balance the competing objectives of achieving statistical convergence (avoiding undershooting at all costs) and maintaining computational efficiency (minimizing unnecessary overshooting). The dramatic disparity in reward magnitude between these two error modes provides a strong inductive bias that guides the agent toward conservative, reliability-prioritizing policies during the early stages of learning.

```python
def _terminate(self) -> Tuple[np.ndarray, float, bool, Dict]:
    error = self.current_shots - self.optimal_shots

    if error < 0:
        # SEVERE PENALTY FOR UNDERSHOOTING
        final_reward = -1.0 - (abs(error) / self.optimal_shots)
    else:
        # DECAYING REWARD FOR OVERSHOOTING
        final_reward = np.exp(-0.0005 * error)

    # ... (rest of the function)
```

Code 4.16: The asymmetric reward function in the `_terminate` method. This function shapes the agent's policy by heavily penalizing undershooting while rewarding efficient, accurate stops.

4.5.3 Procedural steps: the DQN learning cycle

The training process is orchestrated by the `train_agent` function (Code 4.17), which implements the standard Deep Q-Learning algorithm with experience replay as explained in Section 2.3.6. This function manages the iterative interaction between the agent and environment over thousands of episodes, gradually refining the agent's policy through repeated exposure to diverse quantum circuit optimization problems.

The process for each training episode unfolds according to the following sequence:

1. **Reset:** the environment is reset by invoking `env.reset()`, which performs several initialization steps: (1) it selects a new quantum problem triplet (algorithm, circuit size, backend) from the dataset, with a 70% probability of choosing a hard problem (requiring > 5000 shots) to focus learning on challenging cases; (2) it retrieves the pre-computed optimal shot count for this problem from the oracle cache; (3) it resets all episode-specific counters (`current_shots` to 0, `outcome_history` to empty); and (4) it returns an 8-dimensional initial state vector encoding the problem's characteristics.

2. **Act:** the agent selects an action using its ε-greedy exploration strategy implemented in `agent.act(state)`. With probability ε (starting at 1.0 and decaying to 0.01), the agent explores by choosing a random action. Otherwise, it exploits its current knowledge by selecting the action with the highest Q-value according to its neural network: $a = \arg\max_{a'} Q(s, a'; \theta)$. The action space consists of two discrete choices: action 0 ('CONTINUE' - execute another batch of shots) or action 1 ('STOP' - terminate execution and receive the terminal reward).

3. **Step:** the environment executes the selected action through `env.step(action)`, which triggers different behaviors depending on the action. If the agent chose 'CONTINUE', the environment: (1) increments `current_shots` by the step size (50 shots); (2) queries the qsimbench library to simulate executing this batch on the quantum backend and collects the measurement outcomes; (3) appends these

outcomes to `outcome_history` for computing distribution-based state features; (4) assigns a small penalty reward of -0.02 to encourage efficiency; and (5) computes and returns the updated state vector. If the agent chose 'STOP' (or the maximum shot limit is reached), the environment invokes the `_terminate` method to calculate the final asymmetric reward based on the error between `current_shots` and `optimal_shots`, and returns the terminal state with a `done` flag set to `True`.

4. **Remember:** the agent stores the complete experience tuple $(s_t, a_t, r_t, s_{t+1}, d_t)$ in its replay memory buffer by calling `agent.remember()`. As previously explained, this memory serves as a dataset for breaking temporal correlations in the training data.

5. **Replay (learn):** The agent improves its Q-network by calling `agent.replay(batch_size=64)`. This learning step works as follows: (1) randomly select 64 past experiences from the replay memory; (2) for each experience, calculate what the Q-value should be (the "target" using the Bellman equation and the target network; (3) get the Q-network's current prediction $Q(s_i, a_i; \theta)$ for comparison; (4) measure the error between the prediction and target using mean squared error; and (5) adjust the network's weights via backpropagation to reduce this error. After each learning step, the exploration rate ε decreases and makes the agent gradually rely more on what it has learned rather than random exploration.

6. **Loop:** steps 2-5 are repeated in sequence, with the agent continuously interacting with the environment and learning from experiences, until the episode terminates. Termination occurs when either: (1) the agent voluntarily chooses the 'STOP' action, triggering the terminal reward calculation; or (2) the cumulative shot count reaches the predefined maximum limit of 20,000 shots, forcing automatic termination. The total episode reward, accumulated across all timesteps is recorded in `rewards_history` for monitoring training progress, shown in the next chapter.

```
def train_agent(num_episodes: int = 3000, update_target_every: int =
    500) -> Tuple:
    load_oracle_cache()  # Load pre-computed optimal shots from disk
```

```
    env = IterativeQuantumEnv(hard_problem_bias=0.7)  # 70% hard problems
    agent = Agent(state_size=8, action_size=2)
    rewards_history = []

    for episode in tqdm(range(num_episodes), desc="Training Episodes"):
        state = env.reset()  # Initialize new problem
        done = False

        while not done:  # Episode loop
            action = agent.act(state)  # Select action (epsilon-greedy)
            next_state, reward, done, info = env.step(action)  # Execute action
            agent.remember(state, action, reward, next_state, done)  # Store experience
            agent.replay(batch_size=64)  # Learn from random batch
            state = next_state

        if episode % update_target_every == 0 and episode > 0:
            agent.update_target()  # Sync target network

    save_oracle_cache()  # Persist oracle results
    return agent, env, rewards_history
```

Code 4.17: The main training loop, orchestrating the agent-environment interaction and learning process over 3,000 episodes with periodic target network updates.

5. Validation

This chapter presents a comprehensive evaluation of the Deep Reinforcement Learning (DRL) agent developed for adaptive quantum circuit shot allocation. We begin by describing the experimental framework, including the dataset, evaluation metrics, and computational infrastructure used for validation. We then present experimental results across multiple training configurations, analyzing the agent's learning dynamics and performance characteristics through detailed visualization. Finally, we provide a critical discussion of the results, examining both the strengths and limitations of the proposed approach.

5.1 Experimental Framework

5.1.1 Dataset and problem space

The agent's training and evaluation were conducted using the **QSimBench** library [2], which provides a standardized benchmark suite of quantum circuits with realistic noise models. QSimBench is an execution-level benchmark specifically designed to support reproducible and scalable evaluation of Quantum Software Engineering tools by providing over 20 million precomputed shot-level execution traces. This library offers a diverse collection of quantum algorithms, circuit sizes, and simulated backend configurations that mirror the characteristics of contemporary NISQ-era quantum hardware.

The complete problem space consists of **triplets** of the form (algorithm, circuit_size, backend), where:

- **Algorithms** include: Deutsch-Jozsa (dj), GHZ state preparation (ghz), Quantum Approximate Optimization Algorithm (qaoa), Variational Quantum Eigensolver (vqe), Quantum Fourier Transform (qft), Grover's search algorithm (grover), W State, Quantum Phase Estimation (qpe), Quantum Walk (qwalk), and Quantum Neural Networks (qnn), among others.

- **Circuit sizes** range from 4 to 15 qubits, capturing problems of varying computational complexity.
- **Backends** simulate realistic noise profiles from IBM Quantum hardware, including fake_fez, fake_kyiv, fake_lima, fake_marrakesh, fake_sherbrooke, and fake_torino, as well as the ideal noise-free aer_simulator.

This combination generates a dataset of several hundred unique quantum circuit execution scenarios, each with different convergence characteristics and optimal shot requirements as determined by the Incremental Execution Oracle framework. Each (algorithm, size, backend) configuration in QSimBench includes 20,000 measurement outcomes systematically organized into batches of 50 shots, along with the original circuit definition in OpenQASM format, the backend's noise model, and comprehensive configuration metadata.

Figure 5.1 illustrates the hierarchical structure of the dataset JSON files used by the QSimBench library.

```
"metadata": {
    "shots": {
        "shots_block": 50,
        "total_shots": 20000,
        "shots_performed": [
            50,
            100,
            150,
            ...
        ]
    }
    ...
}
```

Shot Config

```
"metadata": {
    "shots": {},
    "iterations": 400,
    "circuit": "grover",
    "circuit_size": 6,
    "backends": {
        "local": [
            "fake_kyiv"
        ]
    }
}
```

Experiment Meta

```
"ground_truth": {
    ...
    "00110010": 1,
    "01011000": 2,
    "01011001": 2,
    ...
},
"data": [
    ...
    {
        "000010": 1,
        "000011": 2,
        "011010": 47
    }
    ...
]
```

Measurement Data

Figure 5.1: Snapshots of the JSON dataset structure: Shot configuration, Experiment metadata, and Measurement data.

The dataset fields are defined as follows:

`shots`: Contains the configuration for execution volumes.

- `shots_block`: The batch size per iteration (set to 50 shots).
- `total_shots`: The maximum shot budget (20,000 shots), serving as the baseline for comparison.
- `shots_performed`: An array tracking cumulative shots after each iteration (e.g., $[50, 100, \ldots, 20000]$).

`iterations`**:** The total number of algorithmic steps performed ($20,000$ shots$/50$ batch size $=$ 400).

`circuit_size`**:** The number of qubits in the quantum circuit.

`backends`**:** Specifies the execution environment. The value `local:[fake_kyiv]` indicates a local simulator mimicking the noise profile of the IBM "Kyiv" device.

`ground_truth`**:** The baseline probability distribution accumulated after the full 20,000 shots. High-frequency keys (e.g., `"011010":19689`) represent the target states (e.g., marked items in Grover's algorithm).

`policy`/`seed`**:** Experiment parameters ensuring reproducibility.

`data`**:** An array of objects, where each object contains the measurement results for a single 50-shot batch.

- **Keys:** N-bit binary strings representing measurement outcomes (where N is the circuit size).
- **Values:** Integer frequency counts indicating how often an outcome was measured within that specific batch.

5.1.2 Data handling and sampling strategy

A critical challenge in training the agent was addressing the natural imbalance in the problem distribution. The dataset exhibits a heavy skew toward "easy" problems that converge with relatively few shots (< 5000), while "hard" problems requiring extensive sampling (> 5000 shots) are comparatively rare. If left unaddressed, this imbalance would bias the agent toward strategies optimized only for simple cases, severely limiting its utility for computationally challenging quantum circuits.

To mitigate this, I implemented a **difficulty-biased sampling strategy** during training. All problems were categorized based on their oracle-determined optimal shot count, with a threshold of 5000 shots separating easy and hard instances. During each training episode,

the environment selected problems from the hard category with 70% probability, ensuring the agent received substantial exposure to challenging convergence scenarios. This oversampling technique proved essential for learning robust policies that generalize across the full spectrum of problem difficulties.

The QSimBench library's flexible API facilitated this sampling strategy through its `get_outcomes()` function, which supports both sequential and randomized sampling modes with exact shot count control and reproducible seeding, critical features for ensuring experimental reproducibility and fair policy evaluation.

5.1.3 Evaluation metrics

The agent's performance was assessed using quantitative metrics that directly align with the thesis objectives: achieving **computational efficiency** (minimizing shot consumption) while maintaining **statistical reliability** (convergence comparable to the Incremental Execution Oracle). While the current implementation utilizes a Total Variation Distance (TVD) based Oracle as the ground truth, the framework's modular architecture allows these metrics to be adapted for alternative optimality definitions in future work.

The evaluation system generates five interconnected visualizations that collectively characterize the agent's learned policy.

1. Training episodic reward evolution

The cumulative reward G_t obtained in an episode serves as the primary signal for monitoring learning stability during training. It aggregates the step-wise penalties for shot consumption and the terminal reward for stopping accuracy. The reward for a single episode is defined as:

$$G_t = \sum_{k=0}^{T} R_{t+k+1} \tag{5.1}$$

where T is the duration of the episode. To visualize the learning trend, considering the high variance of stochastic exploration, we compute a sliding window moving average:

$$\mu_G[i] = \frac{1}{W} \sum_{j=0}^{W-1} G_{i-j} \tag{5.2}$$

where W is the window size (e.g., 500 episodes). This allows us to distinguish between the initial exploration phase (high volatility) and the stabilization phase (sample example in Figure 5.2).

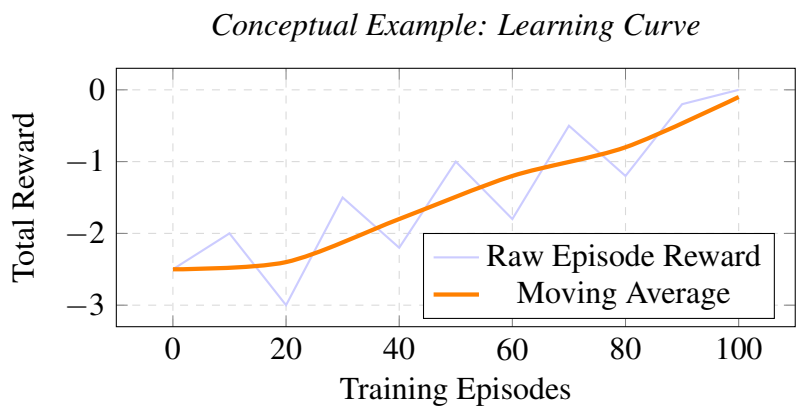

Figure 5.2: Example of reward evolution. The agent improves over time (orange trend) despite the noise inherent in the environment (light blue).

2. Mean Absolute Error (MAE) during training

To quantify the precision of the agent's stopping decisions, we track the Mean Absolute Error (MAE) throughout the training process. The MAE for a specific episode i is the absolute difference between the agent's selected shot count (S_{agent}) and the Oracle's optimal shot count (S_{oracle}):

$$\text{MAE}_i = |S_{\text{agent}}^{(i)} - S_{\text{oracle}}^{(i)}| \tag{5.3}$$

This metric provides a direct measure of how far the agent's policy deviates from the ground truth. A decreasing trend in MAE confirms that the agent is not just maximizing a scalar reward, but is specifically learning to approximate the Oracle's convergence criteria (sample example in Figure 5.3).

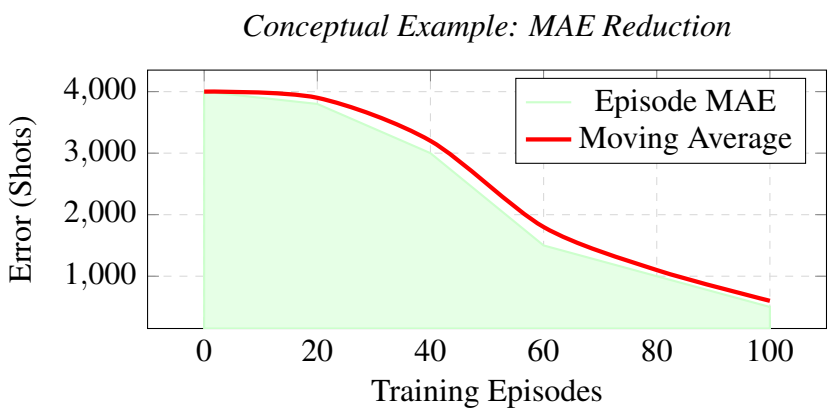

Figure 5.3: Example of MAE progression. A downward trend indicates the agent is learning to stop closer to the optimal point.

3. Policy alignment (scatter analysis)

To evaluate the final trained policy, we map the agent's decisions against the ground truth on a 2D plane (Figure 5.4). For a set of test circuits, we plot points (x, y) where $x = S_{\text{oracle}}$ and $y = S_{\text{agent}}$.

- **Perfect Policy:** Points lie exactly on the diagonal $y = x$.
- **Overshooting (conservative):** Points lie above the diagonal ($y > x$), indicating the agent used more shots than necessary.
- **Undershooting (risky):** Points lie below the diagonal ($y < x$), indicating premature termination.

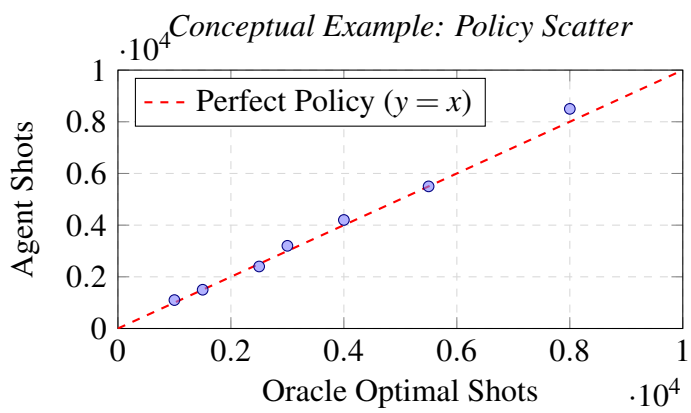

Figure 5.4: Example scatter plot. Points clustering around the diagonal indicate high accuracy; points significantly above indicate systematic overshooting.

4. Distribution of stopping errors

We analyze the distribution of the residuals $E = S_{\text{agent}} - S_{\text{oracle}}$ via a histogram (sample example in Figure 5.5). This visualization reveals the agent's systematic bias and risk profile:

- **Peak at 0:** Indicates high precision.
- **Right-skewed (positive tail):** Indicates a "safety-first" bias (overshooting).
- **Left-skewed (negative tail):** Indicates dangerous undershooting behavior.

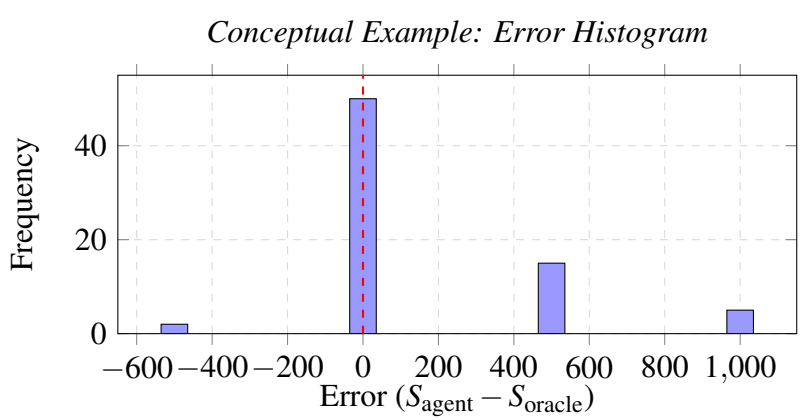

Figure 5.5: Example histogram. Ideally, the mass is centered at 0. A long tail to the right is preferable to a tail to the left in this domain.

5. Error distribution by problem difficulty

Finally, to verify that the agent does not overfit to simple problems, we stratify the error distribution based on problem complexity (Figure 5.6). We partition the test set into two categories:

- **Easy:** $S_{\text{oracle}} \leq 5000$ shots.
- **Hard:** $S_{\text{oracle}} > 5000$ shots.

By overlaying histograms for these two groups, we can detect if the agent's performance degrades on complex circuits (e.g., if hard problems exhibit significantly higher variance or bias than easy ones).

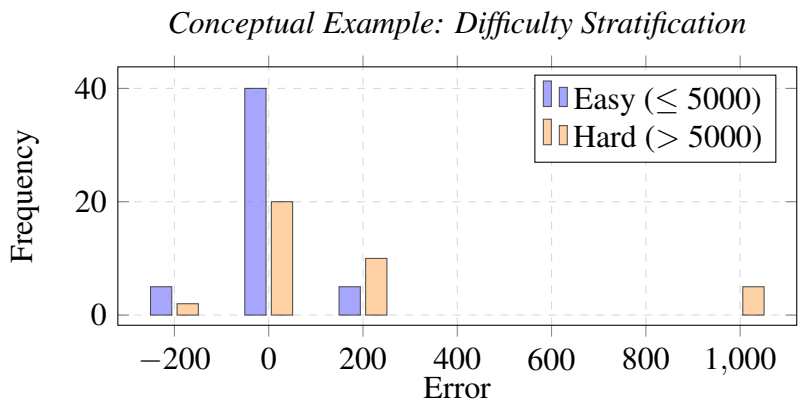

Figure 5.6: Example of stratified errors. This chart helps identify if the agent fails specifically on complex tasks.

5.2 Experimental results

To determine the optimal training duration and assess the agent's sensitivity to training length, we conducted experiments with different episode counts. Each configuration was trained from scratch with identical hyperparameters and identical random seed for reproducibility.

Hyperparameter Configuration

The training process employed the following hyperparameters, selected to balance learning stability with convergence speed:

- **Learning rate** (α): 1×10^{-4} – A conservative value chosen to ensure stable gradient descent updates without overshooting optimal network weights
- **Batch size**: 64 – Provides sufficient sample diversity for stable learning while maintaining computational efficiency
- **Discount factor** (γ): 0.99 – A high value that encourages the agent to consider long-term consequences of its decisions rather than focusing solely on immediate rewards
- **Exploration rate** (ε): Decays from 1.0 to 0.05 – Begins with pure exploration (random actions) and gradually shifts toward exploitation (learned policy), maintaining 5% exploration to prevent policy stagnation

These hyperparameters remained constant across all training configurations to ensure fair comparison between different episode counts.

Model selection phase

To determine the optimal training duration for the agent, we conducted an initial exploratory training run of 10,000 episodes to observe the complete learning trajectory and identify critical transition points. This preliminary investigation proved crucial for understanding the agent's learning trajectory and identifying potential training duration thresholds.

To visualize this trajectory, we analyze the temporal evolution of the two primary training metrics: the episodic reward and the Mean Absolute Error (MAE).

Figure 5.7: Evolution of episodic rewards over 10,000 training episodes. The semi-transparent blue area represents raw episodic rewards, while the orange line shows the 200-episode moving average, revealing underlying learning trends.

The reward signal (Figure 5.7) captures the agent's evolving decision quality through the sum of step-wise efficiency penalties (-0.02 per CONTINUE action) and the terminal reward. The chart reveals a striking pattern: the moving average stabilizes around -2.0 within the first 2000 episodes and maintains this level throughout all 10,000 episodes, yet the variance of individual episode rewards undergoes visible changes. Initially moderate (episodes 0-2000), the variance increases substantially after episode 2000, with rewards spanning from below -3.0 to near $+1.0$. This bifurcation, stable average with increasing variance, indicates the agent is exploring diverse strategies rather than converging to a single policy. Moreover, the fact that the Total Reward per Episode trend doesn't change after roughly 3000 episodes indicates that additional training beyond this point is unlikely

to yield significant improvements.

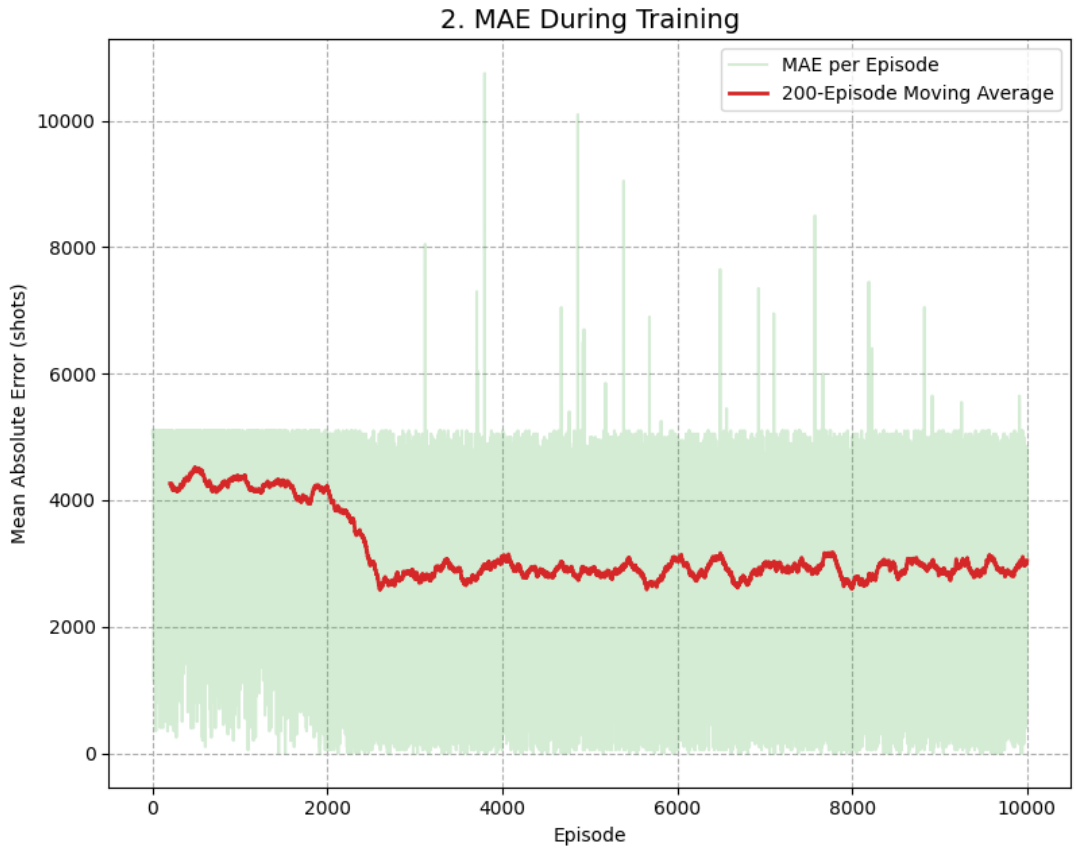

Figure 5.8: Mean Absolute Error during 10,000 training episodes. The semi-transparent green area shows episode-level MAE ($|\text{shots}_{\text{agent}} - \text{shots}_{\text{oracle}}|$), while the red line represents the 200-episode moving average.

The MAE metric (Figure 5.8) provides complementary insight by directly measuring the distance between the agent's stopping decisions and the Oracle's optimal values. Unlike the reward signal, which remained stable in average, the MAE exhibits clear improvement phases. The curve reveals three distinct learning regimes that prove critical for selecting appropriate training durations.

The combined analysis of these metrics reveals the agent's learning trajectory:

1. **Initial stabilization (Episodes 0-2000)**: The reward moving average quickly settles around -2.0 with moderate variance, while the MAE remains elevated above

4000 shots with only gradual decline. This phase represents the agent discovering a conservative baseline policy: stopping execution early to avoid accumulated step penalties. While this "safe" strategy avoids catastrophic failures, it systematically underallocates resources. The ε-greedy exploration rate decays from 1.0 during this period, allowing the agent to exploit early patterns while maintaining exploration.

2. **Critical transition (Episodes 2000-3000)**: Around episode 2000, both metrics undergo correlated transformations. The reward signal exhibits marked increase in volatility (individual episodes ranging from -3.0 to $+1.0$) while maintaining its -2.0 average. Simultaneously, the MAE undergoes a sharp, sustained decline from approximately 4400 shots to 2800 shots, a reduction of over 35%. This bifurcation pattern indicates a fundamental policy breakthrough: the agent overcomes its initial risk-aversion and begins executing longer shot sequences, discovering circuit-specific stopping strategies. The heightened reward variance reflects differentiated responses (success on easy circuits, occasional failures on hard ones) replacing the previous uniform conservatism. Critically, the strong correlation between reward volatility increase and MAE reduction confirms this behavioral shift is productive learning, not random exploration.

3. **Instability emergence (Episodes 3000+)**: Beyond episode 3000, the learning process enters a problematic regime. The MAE moving average stabilizes around 2800-3000 shots and plateaus completely, the curve becomes nearly horizontal despite 7000 additional training episodes. No further improvement occurs. Meanwhile, both the reward and MAE signals show continued increase in episode-to-episode variance, with frequent extreme deviations. This pattern, stagnant average performance coupled with heightened volatility, is the classical signature of overfitting. The agent's experience replay mechanism, now sampling from an increasingly large history (10,000+ episodes), trains on noisy gradients. Rather than refining generalizable convergence patterns, the agent begins learning dataset-specific noise and spurious correlations.

The stabilization of the MAE around episode 3000 suggests this may be a critical inflection

point where productive learning begins to yield to overfitting. Guided by this hypothesis, we selected **three strategic training durations** for validation:

- **2,000 episodes**: To analyze the pre-transition state, serving as a baseline to quantify the performance cost of potential underfitting.
- **3,000 episodes**: To capture the agent immediately following the sharp decline in error, representing the hypothesized peak for generalizable performance.
- **5,000 episodes**: To investigate the effects of moderate extension beyond the stabilization point, testing for early signs of the overfitting dynamics observed in the longer run.

This selection aims to characterize the full learning spectrum, enabling a rigorous analysis of how the agent's policy evolves from an initial baseline to an optimized state, and eventually towards potential degradation.

Evaluation Methodology

After training, each agent was evaluated on a standardized test set of 200 quantum circuit problems uniformly sampled across the problem space, with performance assessed through the five diagnostic charts previously described. All reported training and evaluation times are rounded and based on the previously indicated hardware. Note that while training duration is largely independent of the agent's efficiency, evaluation times fluctuate based on performance; specifically, an agent that frequently overshoots the optimal shot count will run more batches and significantly prolong the execution process.

5.2.1 2,000 episodes configuration

Training Time: $\approx$ 11 min — Evaluation Time: $\approx$ 2 min

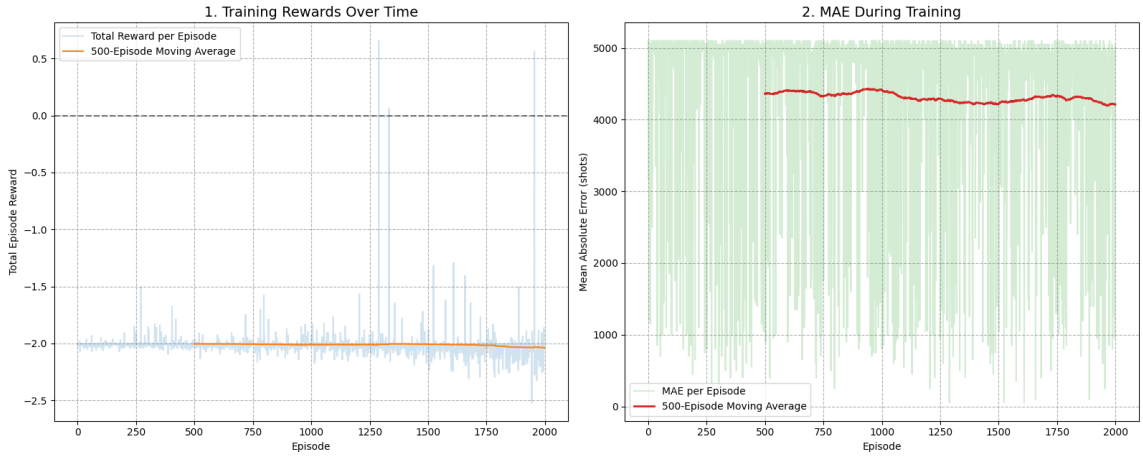

Figure 5.9: Scatter chart of agent performance with 2000 training episodes

The Training Rewards chart shows a moving average that remains effectively flat around -2.0 for nearly the entire duration. This suggests that for the vast majority of the training, the agent was stuck in a local minimum, likely defaulting to immediate or early stopping strategies to avoid accumulated step penalties. It is only toward the very end of the 2000 episodes that we see slight volatility in the raw rewards, hinting that the agent is just beginning to break out of this initial behavior. Consequently, the MAE During Training remains consistently high (above 4000 shots), confirming that the network has not yet converged to a reliable function approximator.

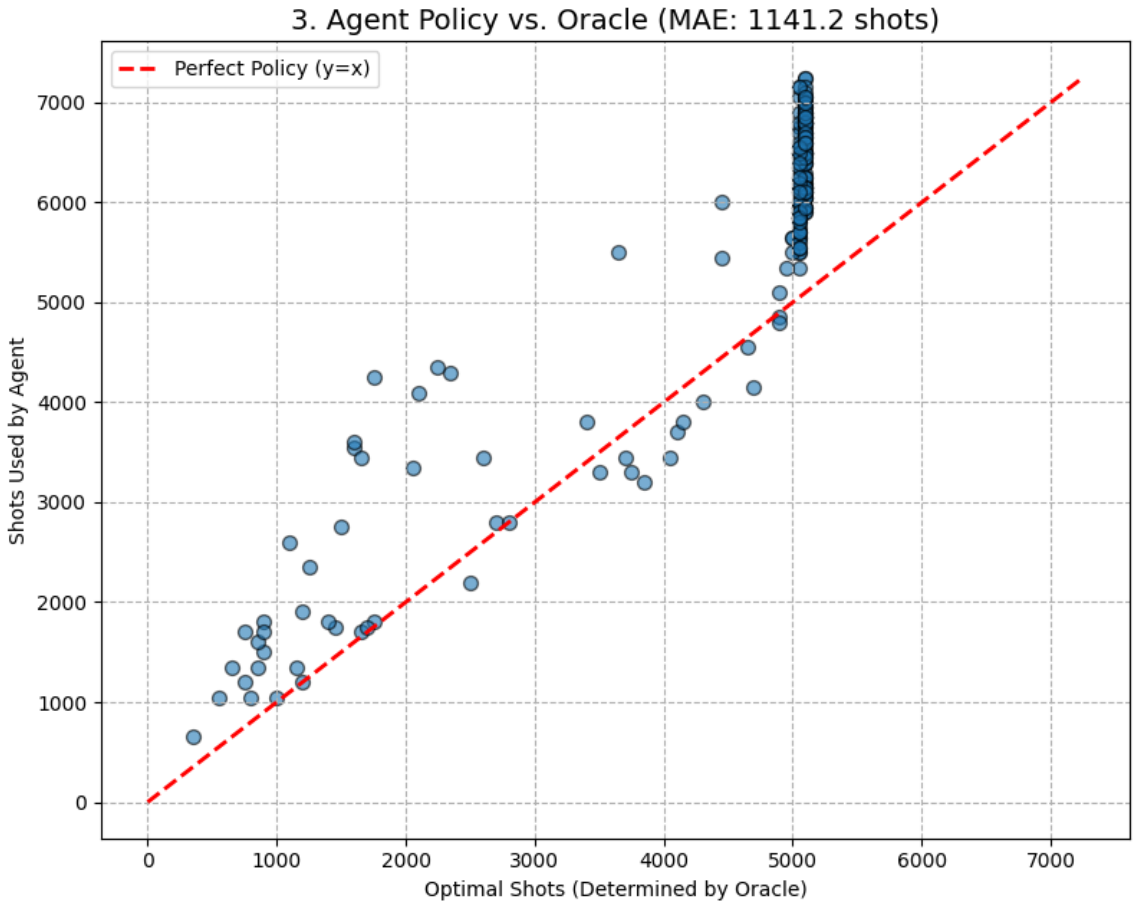

Figure 5.10: Scatter chart of agent performance with 2000 training episodes

The Agent Policy vs Oracle scatter plot (MAE: 1141.2 shots) reveals a disjointed policy. While the agent aligns reasonably well with the Oracle for very simple problems (bottom-left corner), it struggles considerably with complex ones. The vertical cluster of points around the 5000-shot mark on the x-axis indicates that while the agent correctly identifies these problems as "harder", it lacks the precision to determine how hard, resulting in erratic predictions ranging from 5000 to over 7000 shots. Furthermore, there are several points dispersed below the diagonal line, indicating instances of undershooting, though relatively few.

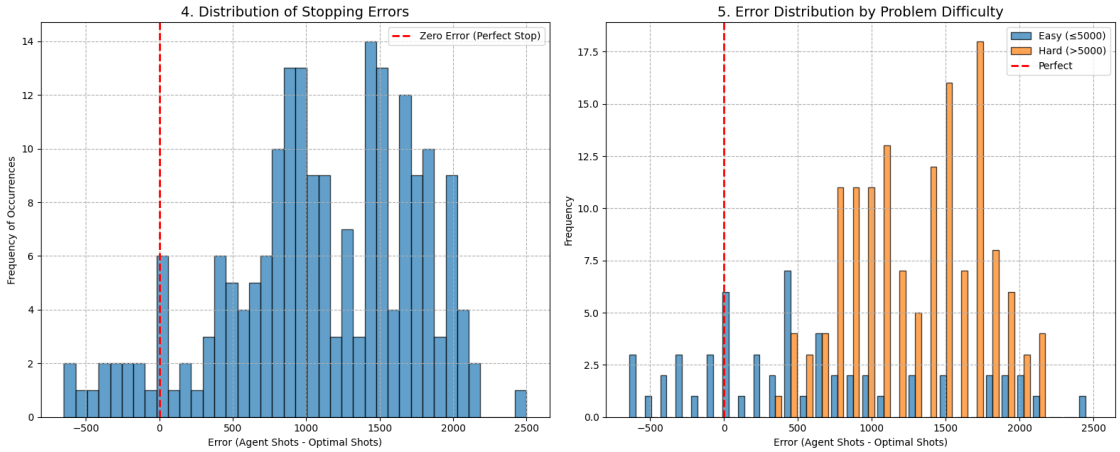

Figure 5.11: Scatter chart of agent performance with 2000 training episodes

The Distribution of Stopping Errors is broad and erratic. Unlike a converged model which typically displays a Gaussian bell curve centered at zero, this distribution is multi-modal and shifted to the right, with a significant number of errors in the +1000 to +2000 range. Chart 5 clarifies that this error is driven largely by the "hard" problems (orange bars), where the agent consistently overestimates the required resources. The presence of errors on both sides of zero (undershooting and overshooting) confirms that at 2000 episodes, the agent has not yet fully internalized the asymmetric risk profile of the reward function.

5.2.2 3,000 episodes configuration

Training Time: $\approx$ 11 min — Evaluation Time: $\approx$ 2 min

The 3000-episode configuration represents the peak performance among the tested durations, demonstrating successful convergence to a robust, generalized policy.

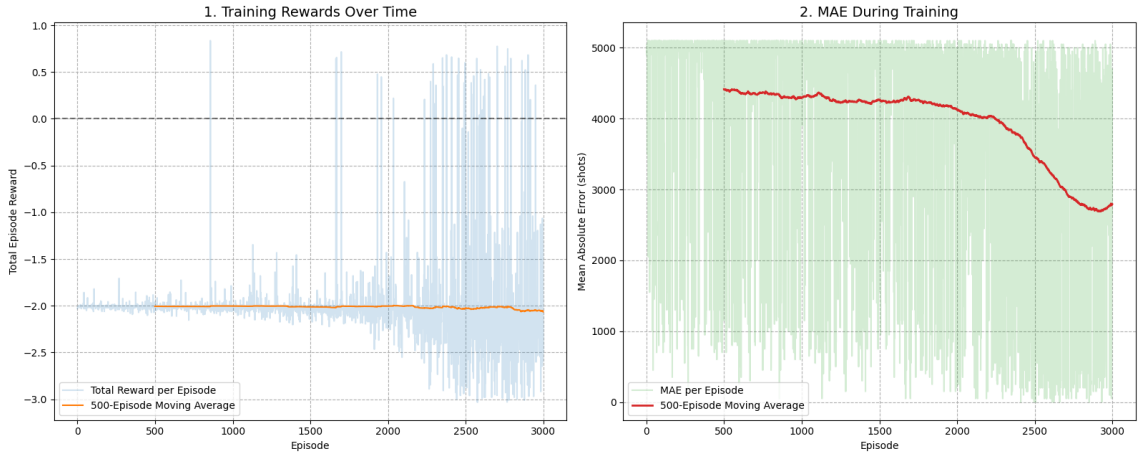

Figure 5.12: Scatter chart of agent performance with 3000 training episodes

The training process exhibits a decisive transition around episode 1600. In the Training Rewards chart, the volatility increases significantly in the final third of the run, while the MAE During Training (Chart 2) shows a sharp, sustained decline from a plateau of ≈ 4400 shots down to ≈ 2800 shots. This confirms that the agent successfully broke out of the initial local minimum (the "immediate stop" strategy) and began actively optimizing its stopping criteria to minimize the distance from the Oracle's ground truth.

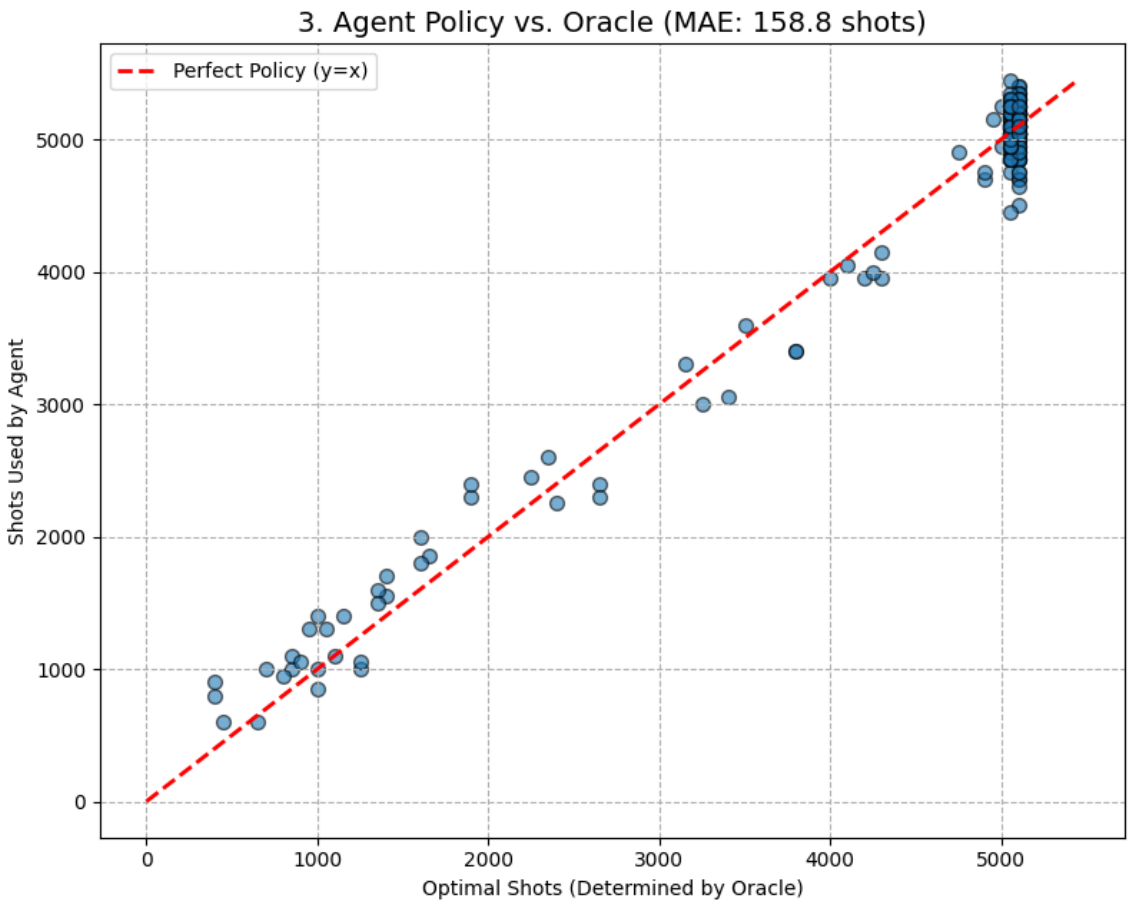

Figure 5.13: Scatter chart of agent performance with 3000 training episodes

The Agent Policy vs Oracle scatter plot shows excellent alignment, with a remarkably low Mean Absolute Error (MAE) of 158.8 shots. Unlike the 2000-episode model, the data points here cling tightly to the $y = x$ diagonal across the entire range of problem difficulties. The agent accurately predicts the stopping point for both simple and complex circuits, indicating that it has successfully learned to interpret the dynamic state features.

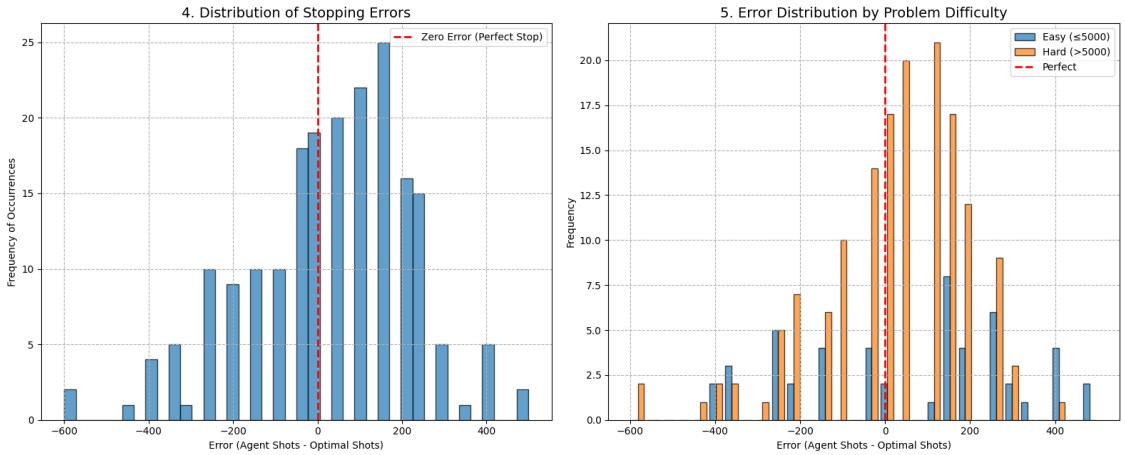

Figure 5.14: Scatter chart of agent performance with 3000 training episodes

The Distribution of Stopping Errors closely resembles a zero-centered Gaussian distribution, which is the ideal outcome for this optimization task. The vast majority of decisions fall within a narrow window of ±200 shots from the optimal.

Crucially, Chart 5 reveals that the agent handles hard problems with nearly the same precision as easy ones. While there is a tendency to overshoot by a small margin, this is a desirable safety feature derived from the asymmetric reward function, as explained in previous sections.

5.2.3 5,000 episodes configuration

Training Time: $\approx$ 20 min — Evaluation Time: $\approx$ 4 min

The 5000-episode configuration reveals the onset of policy degradation due to overfitting, characterized by a significant loss of efficiency compared to the 3000-episode benchmark.

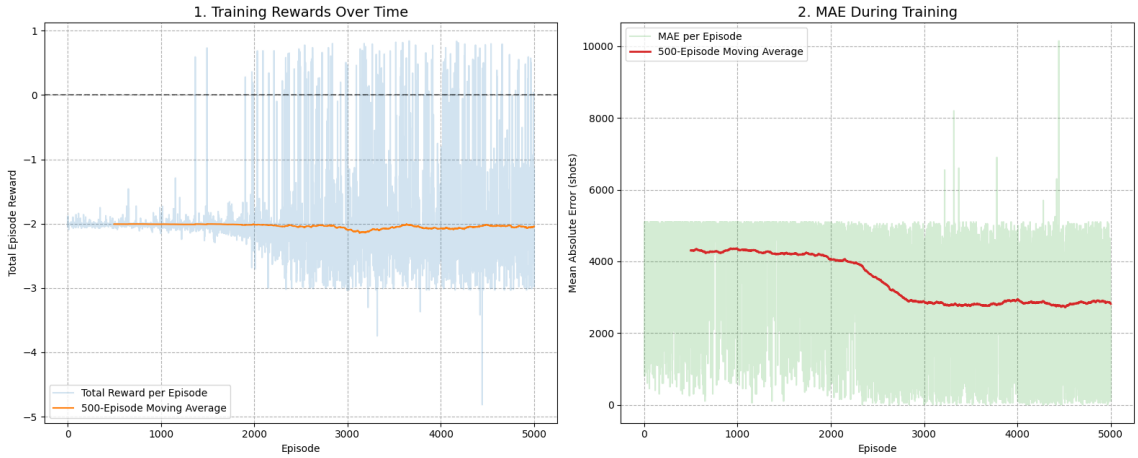

Figure 5.15: Scatter chart of agent performance with 5000 training episodes

The training metrics initially follow the successful trajectory seen in the 3000-episode run, with the Mean Absolute Error (MAE) dropping significantly around episode 2500. However, towards the end of the 5000 episodes, the MAE during training plateaus and does not improve further. The reward signal remains noisy, suggesting that while the agent has learned to avoid immediate failure, it is struggling to refine the delicate balance between stopping early and running longer.

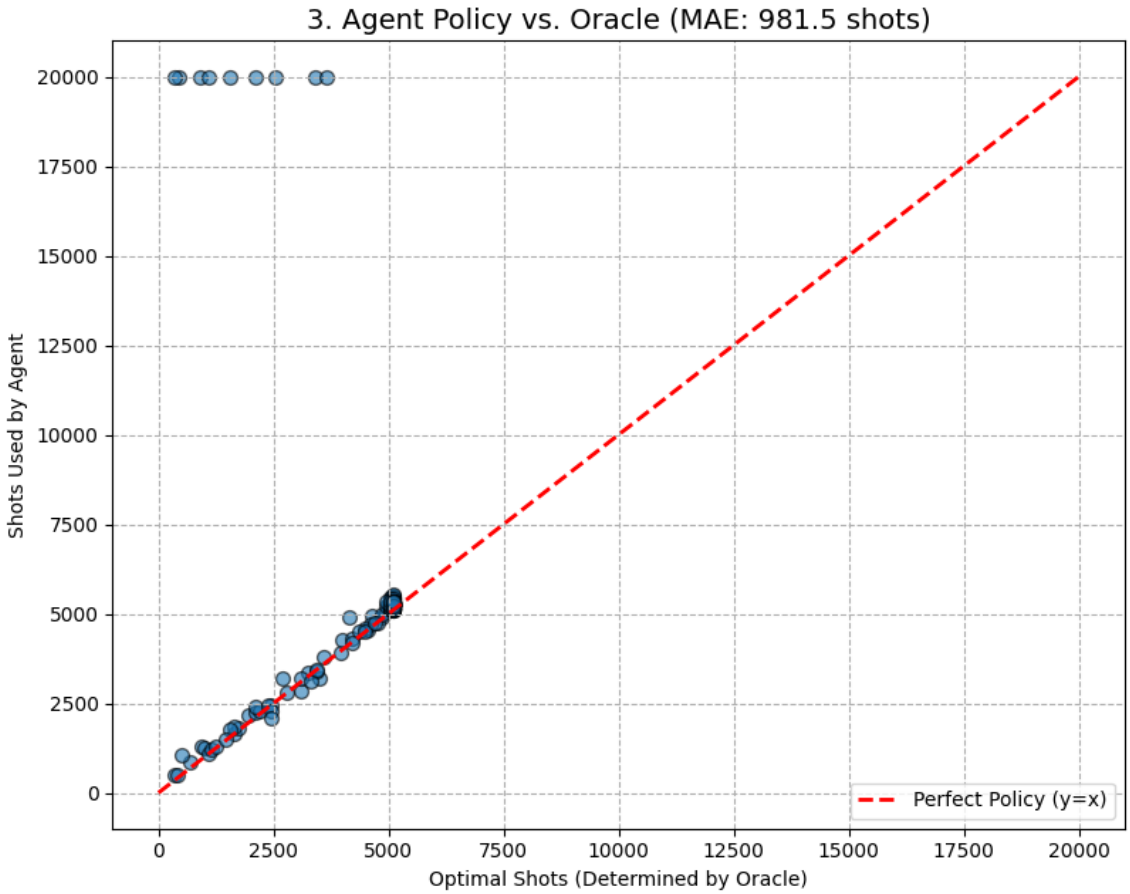

Figure 5.16: Scatter chart of agent performance with 5000 training episodes

The Agent Policy vs Oracle scatter plot illustrates a degradation in performance, with the MAE rising to 981.5 shots. While the majority of points precisely hug the diagonal, a distinct cluster of outliers has appeared at the very top of the graph ($y = 20000$).

These points represent circuits where the Oracle determined only 1,000–4,000 shots were necessary, yet the agent panicked and ran the execution all the way to the maximum budget of 20,000 shots. This "all-or-nothing" behavior indicates that on certain problem instances, the agent has completely lost confidence in its stopping criteria and defaults to the safest possible action (maximum shots) to avoid the undershooting penalty.

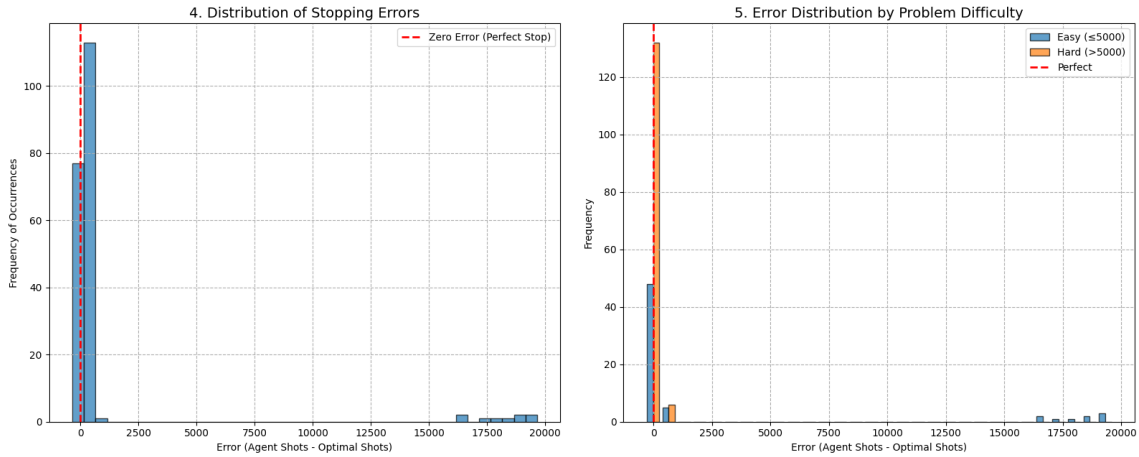

Figure 5.17: Scatter chart of agent performance with 5000 training episodes

The Distribution of Stopping Errors confirms this bifurcation in behavior. While the central peak at zero remains the dominant feature (the agent is still accurate most of the time), a new, small cluster of errors appears at the extreme right edge of the histogram (+16,000 to +19,000 error).

Chart 5 provides the critical insight: these errors are committed primarily on easy problems (blue bars on the far right). This confirms that the agent is overfitting to the risk-averse nature of the reward function. It has learned that for certain ambiguous states, it is "cheaper" to waste 15,000 shots than to risk stopping one shot too early, severely compromising the efficiency goals of the thesis.

5.3 Experiments discussion and critical analysis

The experimental campaign reveals that the performance of the Deep Reinforcement Learning agent is non-monotonic with respect to training duration. By analyzing the trajectory from 2,000 to 10,000 episodes, we have identified that more training does not necessarily equate to better decision-making in this specific domain.

This behavior is clearly illustrated in the comparative analysis presented in Figure 5.18. The scatter plot overlays the policy decisions of four distinct agents trained for different durations. While the 2000-episode agent (red crosses) is scattered and imprecise, and

the 5000 (orange triangles, some are covered by the blue ones) and 10,000-episode (blue squares) agents drift towards conservative overshooting, the 3000-episode agent (green circles) aligns tightly with the optimal diagonal.

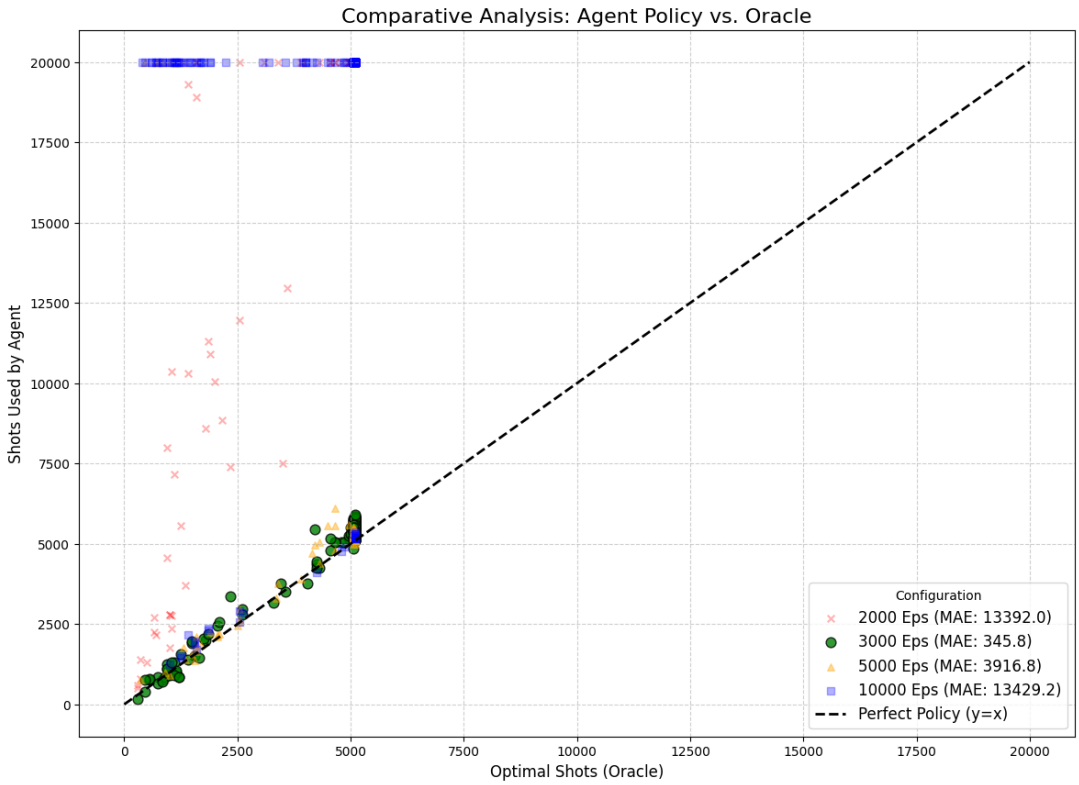

Figure 5.18: Comparative scatter plot overlaying the policy of four agents trained for different durations. The 3000-episode agent (green circles) demonstrates superior alignment with the Oracle's optimal shot count.

5.3.1 The optimal balance: 3000 episodes

The results conclusively point to the **3000-episode** configuration as the most effective agent. With a Mean Absolute Error (MAE) of approximately 158 shots and a Gaussian error distribution centered near zero, this agent demonstrates that it has successfully internalized the nature of the problem.

It is important to contextualize this success with respect to the experimental setup. As the training and evaluation sets were drawn from the same comprehensive problem space,

the agent's high performance confirms that it has mastered the convergence dynamics of the provided dataset. However, this is not merely a case of memorization. Given that quantum measurement outcomes are stochastic—meaning the exact path of the Total Variation Distance (TVD) changes with every execution—the agent cannot simply memorize a scalar "optimal shot count" for a given circuit. Instead, it must have learned to interpret the 8-dimensional state vector, specifically the dynamic relationships between *Distribution Entropy*, *Rate of Change*, and the current shot count. The 3000-episode agent uses these features to correctly identify the point of stability, mimicking the Oracle's logic with good precision.

5.3.2 Critical analysis of policy degradation

A counter-intuitive finding is the significant performance degradation observed at 5,000 and 10,000 episodes. This phenomenon highlights a critical interaction between the agent's learning capacity and the reward function design.

The reward structure was intentionally designed to be asymmetric, but in the extended training sessions (10,000+ episodes), the agent effectively "*overfits*" to this risk profile. It discovers that the mathematical expectation of the cumulative reward is maximized not by being precise, but by being *hyper-conservative*. By defaulting to the maximum shot budget (20,000) whenever a problem presents even slight ambiguity (typical of easy circuits with high noise floors), the agent guarantees it never receives the massive undershooting penalty.

5.3.3 Strengths and behavioral characteristics

Despite the issues at high epoch counts, the proposed approach demonstrates several key strengths:

1. **Robustness to noise:** The agent consistently exhibits a "safety-first" bias. The error distributions are strictly right-skewed, meaning the agent almost never commits

the critical error of stopping too early. In a scientific context, valid (but slightly expensive) results are infinitely preferable to invalid (cheap) results.

2. **Feature efficacy:** The success of the agent validates the hypothesis that the augmented feature set (*Entropy*, *Variance*, *Rate of Change*) provides a sufficient instrument panel for navigation. The agent is not flying blind; it is reacting to the statistical shape of the distribution in real-time.

3. **Modularity:** The agent's policy is derived from observing the state, not from hard-coded heuristics. This implies that the framework respects the modularity constraint of the thesis. If we were to replace the TVD-based Oracle with a different metric, the agent could be retrained to recognize the new convergence signatures without changing the underlying neural network architecture.

6. Conclusions

This thesis has explored the intersection of Quantum Computing and Artificial Intelligence, addressing the critical resource management problem of shot allocation in the Noisy Intermediate-Scale Quantum (NISQ) era. By framing the termination of quantum circuit execution as a sequential decision-making problem, we have demonstrated that a Reinforcement Learning agent can effectively learn to optimize computational resources without compromising statistical validity.

6.1 Summary of Thesis Work

This thesis addressed the challenge of resource optimization in the Noisy Intermediate-Scale Quantum (NISQ) era, specifically focusing on the problem of *shot allocation*, determining the optimal number of circuit executions required to reconstruct a probability distribution with statistical confidence.

The work began with an analysis of the limitations inherent in static allocation strategies and existing heuristic approaches, such as the Incremental Execution (IE) framework. While the Incremental Execution framework introduced adaptivity via rigid convergence thresholds, this research proposed a paradigm shift towards a data-driven, learning-based solution.

This thesis addressed the challenge of resource optimization in the Noisy Intermediate-Scale Quantum (NISQ) era, specifically focusing on the problem of *shot allocation*, determining the optimal number of circuit executions required to reconstruct a probability distribution with statistical confidence.

The work began with an analysis of the limitations inherent in static allocation strategies and existing heuristic approaches, such as the Incremental Execution (IE) framework. While the Incremental Execution framework introduced adaptivity via rigid convergence

thresholds, this research proposed a paradigm shift towards a data-driven, learning-based solution.

The core contribution was the design and implementation of a **Deep Reinforcement Learning (DRL) agent** based on the Deep Q-Network (DQN) architecture. The first developmental step involved **problem formalization**, where we modeled the quantum circuit execution as a sequential Markov Decision Process (MDP). In this formulation, an agent must decide at each step whether to consume more resources (`CONTINUE`) or terminate the process (`STOP`).

A significant portion of the work then focused on **state space engineering** to overcome the agent's initial "blindness" to quantum dynamics. We engineered a sophisticated 8-dimensional state vector functioning as a real-time "instrument panel," incorporating both static context features (circuit size, backend type) and dynamic statistical features such as *Shannon Entropy*, *Distribution Variance*, and the *Rate of Change* to provide the agent with temporal awareness of convergence patterns.

The development of the **environment and Oracle design** constituted another critical component. We constructed a custom simulation environment using the `QSimBench` library to generate realistic execution traces across diverse quantum algorithms and hardware backends. An *Oracle* system, based on the traditional IE framework, was implemented to provide ground-truth supervision for agent evaluation. This setup enabled us to define a specialized **asymmetric reward function**, meticulously calibrated to penalize the scientific failure of undershooting (stopping too early) significantly more severely than the efficiency loss of overshooting. This asymmetric structure proved essential in teaching the agent to prioritize statistical reliability over computational economy.

To address the inherent class imbalance in quantum datasets—where "easy" circuits requiring few shots vastly outnumber computationally demanding "hard" ones—we implemented a specific **training strategy** incorporating difficulty-biased sampling. By forcing the agent to train predominantly on complex problems (70% probability of selecting hard

circuits), we ensured the learned policy developed robustness to handle high-noise scenarios and challenging convergence patterns, rather than merely optimizing for the more common simple cases.

Finally, the work culminated in a rigorous validation campaign spanning multiple training configurations. We trained agents for 2,000, 3,000, 10,000, and 15,000 episodes to systematically analyze learning dynamics and performance evolution. This experimental design allowed us to identify the critical trade-offs between underfitting in short training runs, optimal generalization at intermediate durations, and the overfitting phenomenon observed in extended training regimes, ultimately establishing 3,000 episodes as the optimal balance point for practical deployment.

6.2 Reached objectives and validation of results

The experimental campaign validated the efficacy of the proposed solution, satisfying the objectives outlined in Chapter 1. Through a systematic analysis of training durations ranging from 2,000 to 15,000 episodes, we identified that a training window of **3,000 episodes** yields an optimal balance between generalization and overfitting.

The resulting agent achieved a Mean Absolute Error (MAE) of approximately 158 shots on the test set. Crucially, the validation phase demonstrated that:

- **Accuracy:** The agent aligns almost perfectly with the Oracle on standard problems, stopping execution precisely when statistical convergence is detected.
- **Safety:** On complex, high-noise problems, the agent exhibits a learned conservatism. Rather than risking an invalid result, it defaults to collecting more samples, ensuring that reliability is never sacrificed for speed.
- **Generalization:** The agent successfully learned to interpret the underlying physics of convergence via the dynamic state features, rather than memorizing specific circuit metadata.

Furthermore, the framework proved to be modular. Since the agent learns from the reward signal rather than hard-coded rules, the system remains agnostic to the specific metric used for ground truth. This ensures that the solution can adapt to future changes in how "optimality" is defined in quantum metrology.

6.3 Possible future works

While the Deep Q-Network (DQN) approach proved effective, other Machine Learning paradigms could address specific limitations of the current framework or offer different optimization strategies.

Sequence modeling with transformers and RNNs

The current agent makes decisions based on isolated state observations, effectively treating each step as an independent snapshot without access to the history of previous states. Future implementations could adopt architectures equipped with internal memory, such as *Long Short-Term Memory* (LSTM) networks or *Transformers*. By processing the full timeline of measurement updates rather than a single instant, these models could capture the temporal trajectory of the execution. This context would allow the agent to better distinguish between transient statistical fluctuations (noise) and the genuine stabilization of the probability distribution.

Unsupervised learning for convergence detection

Instead of training an agent to maximize a reward, the problem could be framed as detecting a change in pattern. Unsupervised techniques like *Autoencoders* or *Isolation Forests* could be trained to recognize the statistical behavior of an evolving distribution. The system would then identify "stability" as a deviation from this pattern (an anomaly), triggering a stop signal when the distribution ceases to change significantly, without needing a pre-calculated optimal value from an Oracle.

Direct Prediction via Supervised Learning

To eliminate the overhead of iterative execution entirely, the software could be extended with a Supervised Learning module. A regression model could be trained to predict the total required shots in a "one-shot" manner, based solely on static circuit features and back-end calibration data. This non-RL approach would be less adaptive to runtime anomalies but potentially faster for standard, well-characterized circuits.

Continuous control with policy Gradients

The current agent is limited to a binary choice: stop or add a fixed batch of shots. Algorithms like Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC) support continuous action spaces. This would allow the agent to output the exact number of additional shots needed (e.g., "request 350 more shots") in a single step. This approach would be more efficient than the current fixed-step method, significantly reducing the number of interactions required with the quantum backend.

Multi-Agent Architectures for Hybrid Algorithms

Eventually, if we manage to build an agent focusing on deciding how many shots to run, we could then extend it using a *Multi-Agent* approach. Instead of one agent doing everything, the system would use two: one agent focuses on tuning the circuit to find the right answer, while a second agent manages the budget by deciding how many shots are needed. These two agents would learn to cooperate, balancing the need for high precision with the need to save computing resources.

6.4 Reflections and skills acquired

The development of this thesis provided a unique opportunity to bridge the gap between two cutting-edge fields. From a technical perspective, I acquired deep proficiency in the **PyTorch** framework and the design of custom **OpenAI Gym** environments, alongside a concrete understanding of quantum measurement statistics and NISQ noise models via the

`QSimBench` library.

Methodologically, the challenges encountered—specifically the "reward hacking" phenomenon and data imbalance—underscored the critical importance of rigorous reward engineering and data curation in machine learning workflows.

More broadly, the experience provided an in-depth and comprehensive understanding of the life cycle of a research project related to Quantum Machine Learning. Starting from the analysis and theoretical study of materials, to the analysis of requirements, to the implementation and creation of a working software application, this journey constituted a valuable set of technical and methodological skills. These foundations allow me to develop new interests in these areas and tackle them with greater awareness and preparation in view of more complex future challenges, while also providing a practical taste of what it is like to work in academic research.

In conclusion, this work demonstrates that AI-driven control systems can effectively optimize the operation of quantum hardware, paving the way for smarter, more autonomous quantum computing stacks.

Bibliography

[1] Morteza Aghaee, Arun Akkala, Zulfi Alam, Rizwan Ali, Alejandro Alcaraz Ramirez, Mariusz Andrzejczuk, Andrey E Antipov, Pavel Aseev, Mikhail Astafev, Bela Bauer, Jonathan Becker, Srini Boddapati, Frenk Boekhout, Jouri Bommer, Esben Bork Hansen, Tom Bosma, Leo Bourdet, Samuel Boutin, Philippe Caroff, Lucas Casparis, Maja Cassidy, Anna Wulf Christensen, Noah Clay, William S Cole, Fabiano Corsetti, Ajuan Cui, Paschalis Dalampiras, Anand Dokania, Gijs de Lange, Michiel de Moor, Juan Carlos Estrada Saldaña, Saeed Fallahi, Zahra Heidarnia Fathabad, John Gamble, Geoff Gardner, Deshan Govender, Flavio Griggio, Ruben Grigoryan, Sergei Gronin, Jan Gukelberger, Sebastian Heedt, Jesús Herranz Zamorano, Samantha Ho, Ulrik Laurens Holgaard, William Hvidtfelt Padkær Nielsen, Henrik Ingerslev, Peter Jeppesen Krogstrup, Linda Johansson, Jeffrey Jones, Ray Kallaher, Farhad Karimi, Torsten Karzig, Evelyn King, Maren Elisabeth Kloster, Christina Knapp, Dariusz Kocon, Jonne Koski, Pasi Kostamo, Mahesh Kumar, Tom Laeven, Thorvald Larsen, Kongyi Li, Tyler Lindemann, Julie Love, Roman Lutchyn, Michael Manfra, Elvedin Memisevic, Chetan Nayak, Bas Nijholt, Morten Hannibal Madsen, Signe Markussen, Esteban Martinez, Robert McNeil, Andrew Mullally, Jens Nielsen, Anne Nurmohamed, Eoin O'Farrell, Keita Otani, Sebastian Pauka, Karl Petersson, Luca Petit, Dima Pikulin, Frank Preiss, Marina Quintero Perez, Katrine Rasmussen, Mohana Rajpalke, Davydas Razmadze, Outi Reentila, David Reilly, Richard Rouse, Ivan Sadovskyy, Lauri Sainiemi, Sydney Schreppler, Vadim Sidorkin, Amrita Singh, Shilpi Singh, Sarat Sinha, Patrick Sohr, Tomaš Stankevič, Lieuwe Stek, and et al. Inas-al hybrid devices passing the topological gap protocol, 2022.

[2] Antonio Brogi, Alessandro Bocci, and Giuseppe Bisicchia. Incremental execution for reducing quantum circuit execution shots. arXiv preprint, 2023.

[3] Colin D. Bruzewicz, John Chiaverini, Robert McConnell, and Jeremy M. Sage.

Trapped-ion quantum computing: Progress and challenges. *Applied Physics Reviews*, 6(2), 2019. arXiv preprint arXiv:1904.04178.

[4] Ronald de Wolf. Quantum computing: Lecture notes. arXiv preprint, 2023. arXiv:1907.09415v5 [quant-ph].

[5] David P. DiVincenzo. Quantum gates and circuits, 1997.

[6] Laleh Aghababaie-Beni Georg Aigeldinger Ross Alcaraz Igor Aleiner Trond I. Andersen Markus Ansmann Frank Arute Kunal Arya Abraham Asfaw Nikita Astrakhantsev Juan Atalaya Ryan Babbush Dave Bacon Brian Ballard Joseph C. Bardin Andreas Bengtsson Alexander Bilmes Sergio Boixo Gina Bortoli Alexandre Bourassa Jenna Bovaird Dylan Bowers Leon Brill Michael Broughton David A. Browne Brett Buchea Bob B. Buckley David A. Buell Tim Burger Brian Burkett Nicholas Bushnell Anthony Cabrera Juan Campero Hung-Shen Chang Yu Chen Zijun Chen Ben Chiaro Liang-Ying Chih Desmond Chik Charina Chou Jahan Claes Agnetta Y. Cleland Josh Cogan Roberto Collins Paul Conner William Courtney Alexander L. Crook Ben Curtin Sayan Das Laura De Lorenzo... Dmitry A. Abanin, Rajeev Acharya. Observation of constructive interference at the edge of quantum ergodicity. *Nature*, 2025.

[7] Hermann Edlbauer, Junliang Wang, A. M. Saffat-Ee Huq, Ian Thorvaldson, Michael T. Jones, Saiful Haque Misha, William J. Pappas, Christian M. Moehle, Yu-Ling Hsueh, Henric Bornemann, Samuel K. Gorman, Yousun Chung, Joris G. Keizer, Ludwik Kranz, and Michelle Y. Simmons. An 11-qubit atom processor in silicon, 2025.

[8] Albert Einstein, Boris Podolsky, and Nathan Rosen. Can quantum-mechanical description of physical reality be considered complete? *Physical Review*, 47(10):777–780, 1935.

[9] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6–7):467–488, 1982.

[10] R. Guzmán, J. C. Retamal, J. L. Romero, and C. Saavedra. Entanglement rate in qubits. *Physics Letters A*, 323(5):382–388, 2004.

[11] He-Liang Huang, Dachao Wu, Daojin Fan, and Xiaobo Zhu. Superconducting quantum computing: A review. *Science China Information Sciences*, 63(8), 2020. arXiv preprint arXiv:2006.10433.

[12] Yuwei Jin, Xiangyu Gao, Minghao Guo, Henry Chen, Fei Hua, Chi Zhang, and Eddy Z. Zhang. Quantum fourier transformation circuits compilation, 2023.

[13] Ville Lahtinen and Jiannis K. Pachos. A short introduction to topological quantum computation, 2017.

[14] Yuxi Li. Deep reinforcement learning: An overview, 2017. arXiv preprint arXiv:1701.07274.

[15] MathWorks. Reinforcement learning: A brief guide. `https://www.mathworks.com/company/technical-articles/reinforcement-learning-a-brief-guide.html`, 2023. Accessed: 2025-10-22.

[16] MathWorks. What is reinforcement learning? `https://www.mathworks.com/discovery/reinforcement-learning.html`, 2023. Accessed: 2025-10-22.

[17] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[18] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, Cambridge, UK, 10th anniversary edition, 2010.

[19] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.

[20] ResearchGate. Dqn architecture: main network, target network, and replay memory, 2023.

[21] Jacquiline Romero and Gerard Milburn. Photonic quantum computing, 2024.

[22] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2020. Online version available at `http://incompleteideas.net/book/the-book-2nd.html`.

[23] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[24] WikiDocs. Introduction to quantum computing, 2023.

[25] Karen Wintersperger, Florian Dommert, Thomas Ehmer, Andrey Hoursanov, Johannes Klepsch, Wolfgang Mauerer, Georg Reuber, Thomas Strohm, Ming Yin, and Sebastian Luber. Neutral atom quantum computing hardware: Performance and end-user perspective. *EPJ Quantum Technology*, 10, 2023.